Software Requirements Specification

for

FamPlan

Version 1.7 - Lab 5 Final Revision

Prepared by Banana Five-O

NTU SC2006 Software Engineering

4 November 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History iii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
1.6 Team Contributions and AI Use 2
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Constraints Assumptions and Dependencies 4
3. External Interface Requirements 4
3.1 User Interfaces 4
3.2 Hardware Interfaces 14
3.3 Software Interfaces 14
3.4 Communications Interfaces 15
4. Functional Requirements 15
4.1 Account Management 15
4.2 Family and Neighbourhood Preferences 15
4.3 Neighbourhood Discovery 16
4.4 Family Amenities and Neighbourhood Information 17
4.5 Map and Travel 17
4.6 Family Decision Support 18
4.7 Automatic Data Synchronisation and Resilience 19
5. Nonfunctional Requirements 19
5.1 Performance Requirements 19
5.2 Scalability Requirements 20
5.3 Usability Requirements 20
5.4 Accessibility Requirements 20
5.5 Compatibility Requirements 20
5.6 Reliability Requirements 21
5.7 Recoverability Requirements 21
5.8 Fault isolation Requirements 21
5.9 Data provenance Requirements 21
5.10 Data freshness Requirements 21
5.11 Reproducibility Requirements 21
5.12 Data quality Requirements 22
5.13 Transport security Requirements 22
5.14 Credential security Requirements 22
5.15 Authorisation Requirements 22
5.16 Authentication security Requirements 22
5.17 Input security Requirements 23
5.18 Privacy Requirements 23
5.19 Data minimisation Requirements 23
5.20 Maintainability Requirements 23
5.21 Testability Requirements 23
5.22 Transparency Requirements 23
5.23 Responsible guidance Requirements 24
5.24 Business Rules 24
6. Data Dictionary 24

7. Use Case Model 26
7.1 Use Case Diagram 26
7.2 Actors 27
7.3 Use Case Catalogue 28
7.4 Functional Requirement Traceability 28
7.5 Use Case Descriptions 30
7.5.1 UC-1.1 Register Account 31
7.5.2 UC-1.2 Login and Logout 32
7.5.3 UC-1.3 Reset Password 33
7.5.4 UC-1.4 Manage Account Details 34
7.5.5 UC-1.5 Delete Account 35
7.5.6 UC-2.1 Manage Family Preferences 36
7.5.7 UC-3.1 Search Neighbourhoods 37
7.5.8 UC-3.2 Apply Saved Preferences 38
7.5.9 UC-4.1 View Neighbourhood Profile 39
7.5.10 UC-4.2 Explore Family Amenities 40
7.5.11 UC-5.1 View Map and Nearby Places 41
7.5.12 UC-5.2 Check Commute Route 42
7.5.13 UC-6.1 Calculate Family Suitability Score 43
7.5.14 UC-6.2 Compare Neighbourhoods 44
7.5.15 UC-6.3 Manage Shortlist and Notes 45
7.5.16 UC-7.1 Monitor Automatic Data Synchronisation 46
8. Technology Stack Recommendation 46
8.1 Architecture Boundary 47
8.2 System Architecture 48
9. Class Diagram 50
9.1 Entity Model 50
9.2 Boundary and Control Subsystems 52
9.3 Robustness View 53
9.4 Design Layer 56
9.5 Design-to-Code Traceability 57
10. Dialog Map 60

Revision History

Name	Date	Reason For Changes	Version
Banana Five-O	8 September 2026	Initial SRS for Lab 1 submission	1.0
Banana Five-O	12 September 2026	Revised SRS for family-neighbourhood scope	1.1
Banana Five-O	5 October 2026	Revised SRS for Lab 3 submission	1.2
Banana Five-O	16 October 2026	Design refinement: 9.4 design layer, 9.5 traceability	1.5
Banana Five-O	31 October 2026	Lab 5 final revision: travel modes, multi-destination commute scoring, exact address search, enlarged architecture figure, updated class diagrams and dialog map, equal team contributions	1.6
Banana Five-O	4 November 2026	Use case diagram, actors and 16 use case descriptions added to Section 7; UI mockups replaced by screenshots of the current build (3.1)	1.7

1. Introduction

1.1 Purpose

This Software Requirements Specification defines FamPlan as a family-neighbourhood decision-support web application. It helps couples and households planning for children compare Singapore neighbourhoods using nearby childcare, kindergartens, primary and secondary schools, supermarkets, healthcare, green spaces, public transport, commute estimates, and accessibility. The product focuses exclusively on family-neighbourhood discovery and comparison. This revision records 49 functional requirements, 29 measurable non-functional requirements, a tabular data dictionary, a 16-use-case model with its diagram and full use case descriptions, traceability, screenshots of the current user interface, and a simple technology recommendation.

1.2 Document Conventions

Functional requirements use shall and have unique FR identifiers. Non-functional requirements use must or must not and have unique NFR identifiers. Must and Should are priority classifications. Visitor is the base public actor. Registered User inherits Visitor behaviour, and Data Administrator inherits Registered User behaviour. Automatic data synchronisation is internal system behaviour; administrators monitor its status and do not manually start updates.

1.3 Intended Audience and Reading Suggestions

The document is intended for the project team, teaching staff, developers, testers, and future maintainers. Sections 1 and 2 define the problem and scope, Section 3 records interfaces, including screenshots of the current build, Sections 4 and 5 are the testable requirement baseline, Section 6 defines shared domain terms, Section 7 gives the use case model (diagram, actors, catalogue, requirement traceability and use case descriptions), and Section 8 proposes a deliberately small implementation stack.

1.4 Product Scope

FamPlan helps a household answer a focused question: which neighbourhood best supports the family's planned life stages and daily needs? Users search neighbourhoods, inspect family-relevant facilities and travel information, adjust transparent weights, compare alternatives, and save a shortlist. The score is an explainable comparison aid rather than an instruction or guarantee. FamPlan is not a property marketplace, valuation or transaction service, school-admission predictor, live childcare-vacancy service, medical adviser, or guarantee that one area suits every family.

1.5 References

Reference	Purpose
SC2006 Lab 1 Manual	Defines requirements, data dictionary, initial use-case model, and UI-mockup deliverables.
SC2006 Project Description	Requires meaningful use of authorised real-world data and clear user value.
FamPlan SRS Lab 1	Previous baseline from which the family-neighbourhood revision was derived.
Configured public datasets	Planned sources for schools, early-childhood services, supermarkets, healthcare, parks, public transport, maps, and routing; exact endpoints and licences must be confirmed before implementation.

1.6 Team Contributions and AI Use

Member	Username	Team Leader	Lab Contributions
POLOYAPOY RON JOSHUA ABES (Apollo)	POLO0002	Yes	Equal share of all labs: requirements, data dictionary, use case model, UI mockups, dialog map, class diagrams, implementation and testing. Star feature: Serverless architecture and load testing.
LIM RUI XIAN EDWARD	ELIM095	No	Equal share of all labs: requirements, data dictionary, use case model, UI mockups, dialog map, class diagrams, implementation and testing. Star feature: Self-refreshing data pipeline from 9 public sources.
LE VIET HOANG ANH (Raphael)	HOANGANH003	No	Equal share of all labs: requirements, data dictionary, use case model, UI mockups, dialog map, class diagrams, implementation and testing. Star feature: Family preferences and multi-destination commute scoring.
ATHENA LIM HUI RU (LIN HUIRU)	ALIM102	No	Equal share of all labs: requirements, data dictionary, use case model, UI mockups, dialog map, class diagrams, implementation and testing. Star feature: Explainable family score, comparison and shortlist.
MUHAMMAD ILHAM BIN IRWAN	MUHA0313	No	Equal share of all labs: requirements, data dictionary, use case model, UI mockups, dialog map, class diagrams, implementation and testing. Star feature: Exact address search, radius and live routes.

AI-use disclosure: OpenAI Codex assisted with consolidating team-authored artifacts, consistency editing, and populating the supplied Word templates; the team reviewed and accepts responsibility for the final content.

2. Overall Description

2.1 Product Perspective

Family-related location information is distributed across separate government and public services. FamPlan consolidates authorised records into neighbourhood profiles and applies transparent, user-controlled criteria. It stores only account, preference, shortlist, note, and synchronisation metadata required by the documented functions. Public source data is synchronised automatically and validated before a new local snapshot becomes active.

2.2 Product Functions

- Manage registration, authentication, password reset, display-name changes, and account deletion.
- Store family stages, preferred areas, commute destinations with travel modes, amenity thresholds, and criterion weights.
- Search and filter neighbourhoods by town, exact address or postal code, rather than property listings.
- Show childcare, education, groceries, healthcare, green spaces, transport, and accessibility evidence.
- Show interactive maps, distances, routes, and commute times with source and retrieval information.
- Calculate explainable family suitability scores and compare two to four neighbourhoods.
- Maintain a ten-neighbourhood shortlist with optional 500-character private notes.
- Synchronise configured public data automatically and let administrators monitor status and history.

2.3 User Classes and Characteristics

User Class	Characteristics and Permissions
Visitor	Can search and inspect neighbourhoods, family amenities, maps, routes, scores, and comparisons. Cannot save personal state.
Registered User	Inherits Visitor functions and can save family preferences, shortlist up to ten neighbourhoods, and maintain private notes.
Data Administrator	Inherits Registered User functions and can view automatic data-source status and synchronisation history. Does not manually trigger synchronisation.
Configured Public Data Source	External supporting system supplying authorised neighbourhood, facility, transport, map, or route information.

2.4 Operating Environment

FamPlan is a responsive browser-based application accessed from modern desktop and mobile browsers. A server-side application maintains accounts, preferences, cached public data, scoring logic, and automatic synchronisation. The system requires network access for current maps, routes, and source updates, while validated cached neighbourhood information remains available during supported provider outages.

2.5 Constraints Assumptions and Dependencies

ID	Assumption or Constraint	Consequence or Control
C-1	Only authorised public APIs or downloadable datasets may be used.	Confirm licences, quotas, schemas, and update frequencies before implementation.
C-2	A neighbourhood needs a documented reference point or boundary basis.	Show the basis and avoid implying that every resident is equally close to each facility.
C-3	Facility proximity does not imply admission, eligibility, vacancy, or service quality.	Display the limitation wherever education or childcare evidence is used.
C-4	Family preferences use stages and priorities, not children's names, birth dates, applications, or medical information.	Apply data minimisation in forms, storage, analytics, and logs.
C-5	Configured sources can differ in coverage and freshness.	Show category-level source, date, missing-data, and stale-data states.
C-6	Automatic source synchronisation runs at least every 24 hours.	Administrators monitor outcomes; failed candidates never replace the last validated snapshot.
A-1	OneMap or an equivalent authorised provider supports mapping, geocoding, and routing.	If coverage is unavailable, only dependent functions show an unavailable state.
A-2	School, early-childhood, retail, healthcare, recreation, and transport sources expose sufficient location data or addresses that can be geocoded.	Unsupported categories must be revised explicitly rather than populated with invented data.

3. External Interface Requirements

3.1 User Interfaces

The interface provides labelled search and preference controls, accessible category filters, a neighbourhood summary, a map with a non-map alternative, facility lists, source and date labels, score explanations, comparison, shortlist, account pages, and an administrator status page. Education and childcare results state that proximity does not guarantee admission or availability. All screens must use the family-neighbourhood vocabulary defined in Section 6.

The screens below are captured from the current FamPlan build (repository main branch, local deployment, 4 November 2026) and replace the Lab 1 mockups. Each caption lists the use cases the screen realises; the screen-to-screen navigation is shown in the dialog map (Section 10).

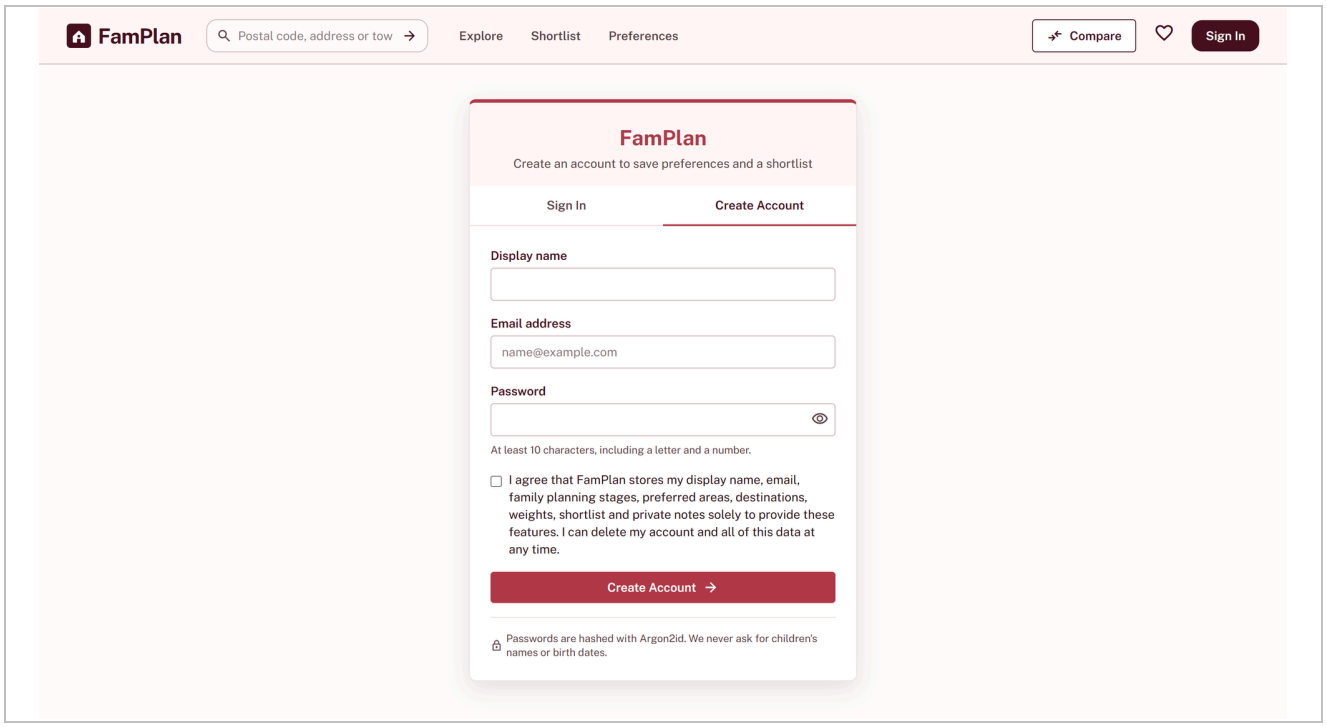


Figure 3.1. Sign in and Create Account (UC-1.1, UC-1.2). Account creation with the data-use consent statement; the Sign In tab uses the same card.

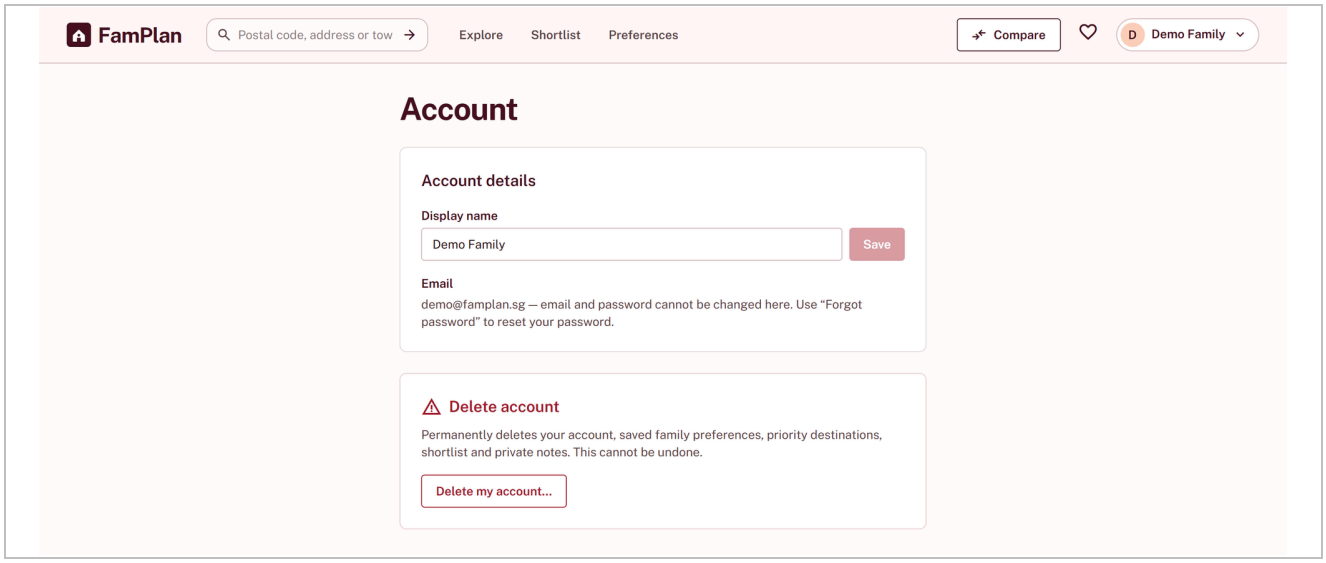


Figure 3.2. Account settings (UC-1.4, UC-1.5). Display-name change (email is read-only) and the account deletion entry point.

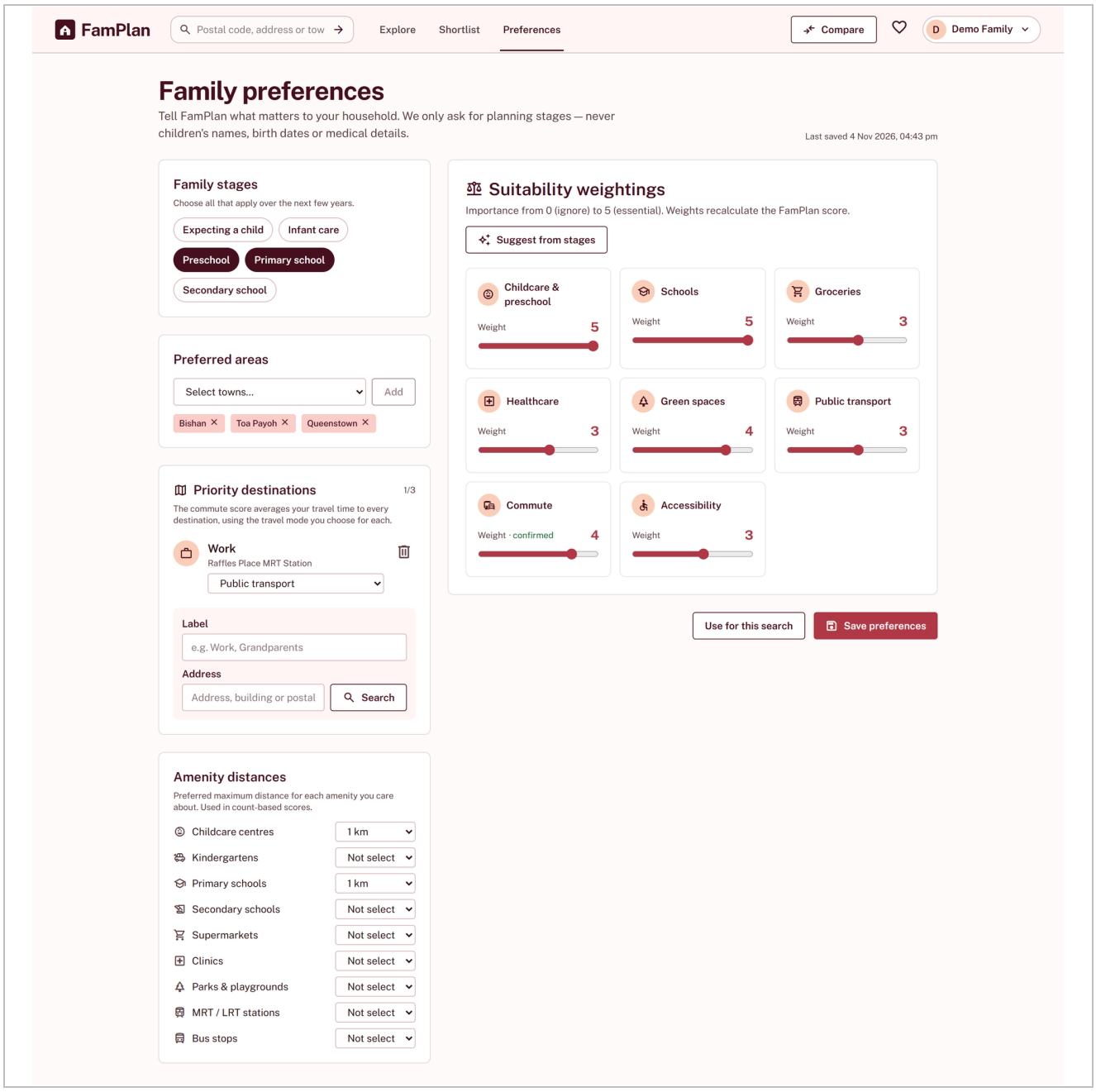


Figure 3.3. Family preferences (UC-2.1). Family stages, preferred areas, up to three priority destinations each with a travel mode, amenity distances and the eight criterion weights.

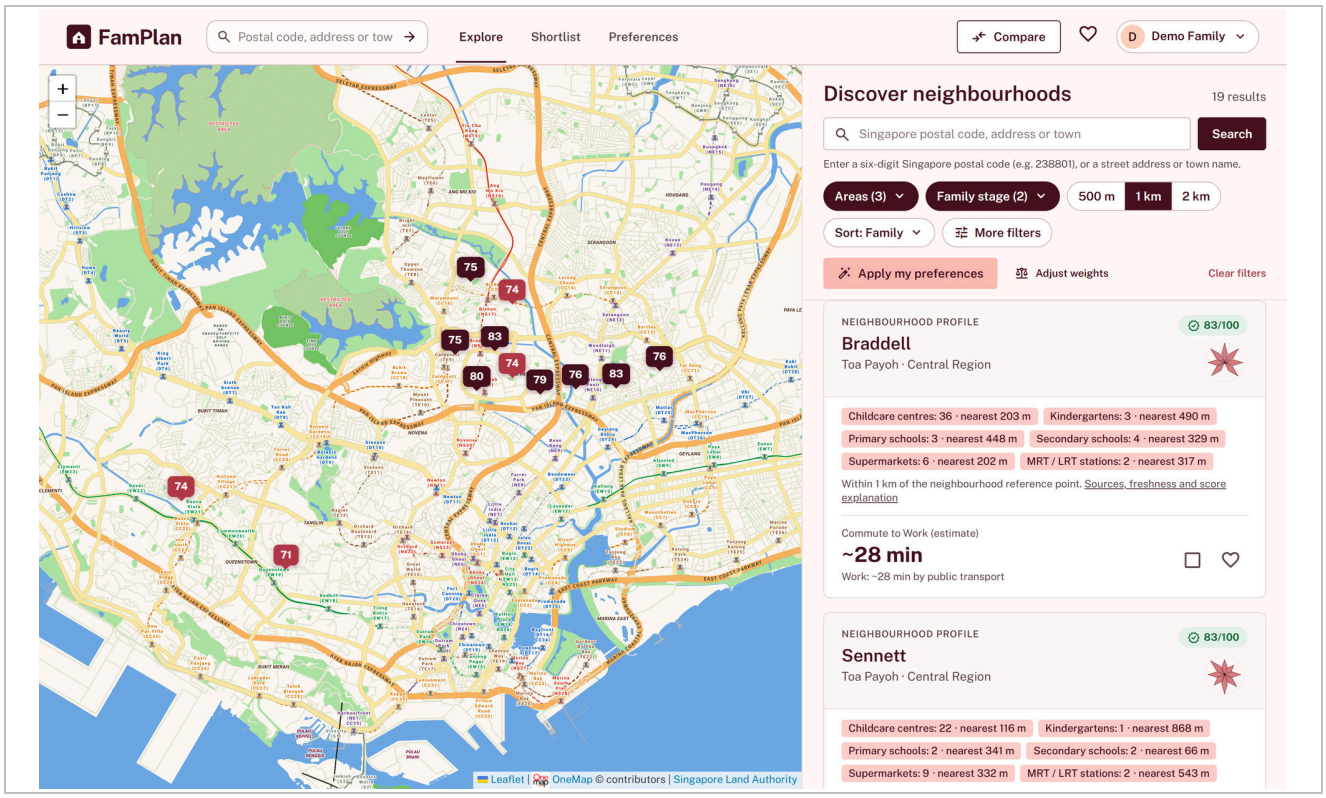


Figure 3.4. Discover neighbourhoods (UC-3.1, UC-3.2). Search, filters and sort, with saved preferences applied. Each result is labelled as a neighbourhood profile and shows its score, amenity counts and commute estimate.

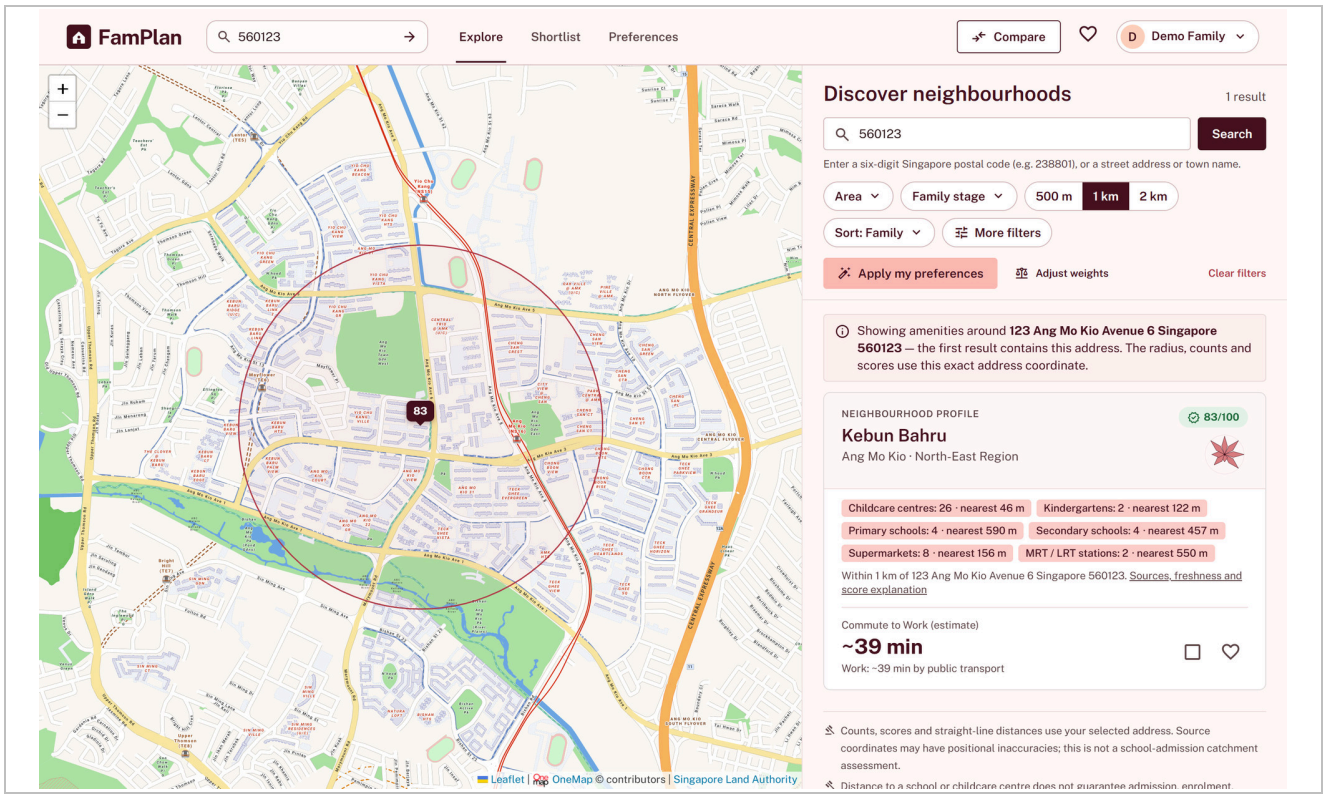


Figure 3.5. Exact address search (UC-3.1 (FR-DISC-07)). A postal code resolved to one exact address; the radius, counts and scores are measured from that coordinate.

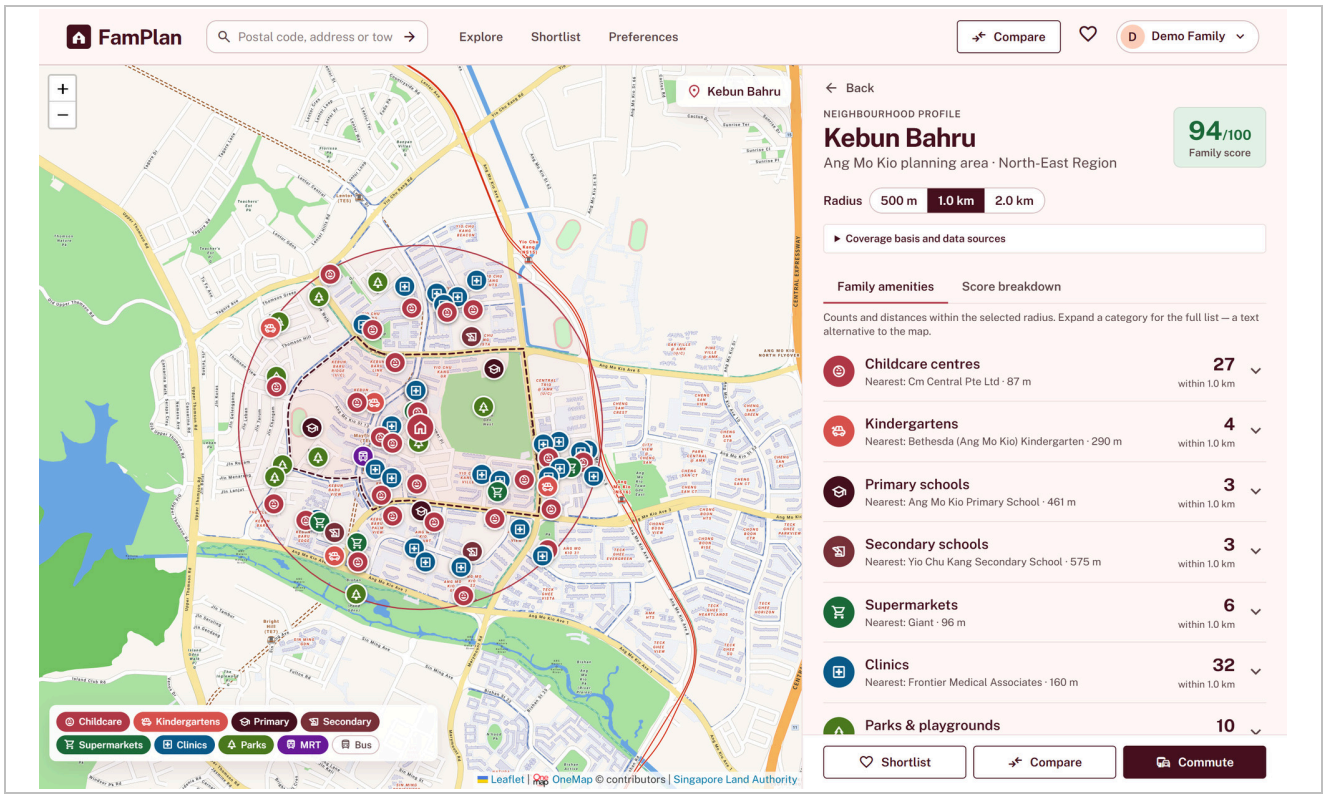


Figure 3.6. Neighbourhood profile: family amenities and map (UC-4.1, UC-4.2, UC-5.1). Family score, radius selector, per-category counts with the nearest facility, and the categorised map with layer toggles.

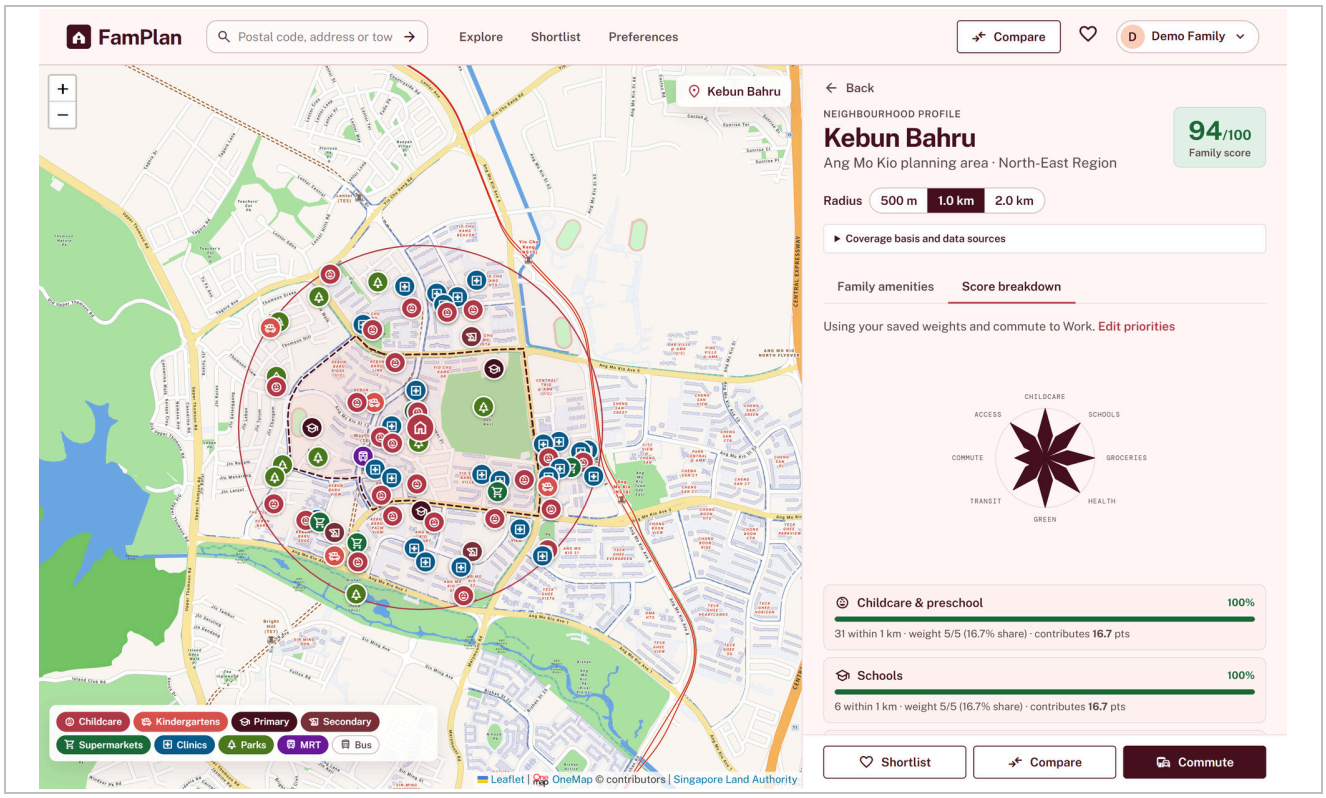


Figure 3.7. Neighbourhood profile: score breakdown (UC-4.1, UC-6.1). Itemised criterion breakdown showing evidence, weight, share and contribution to the total.

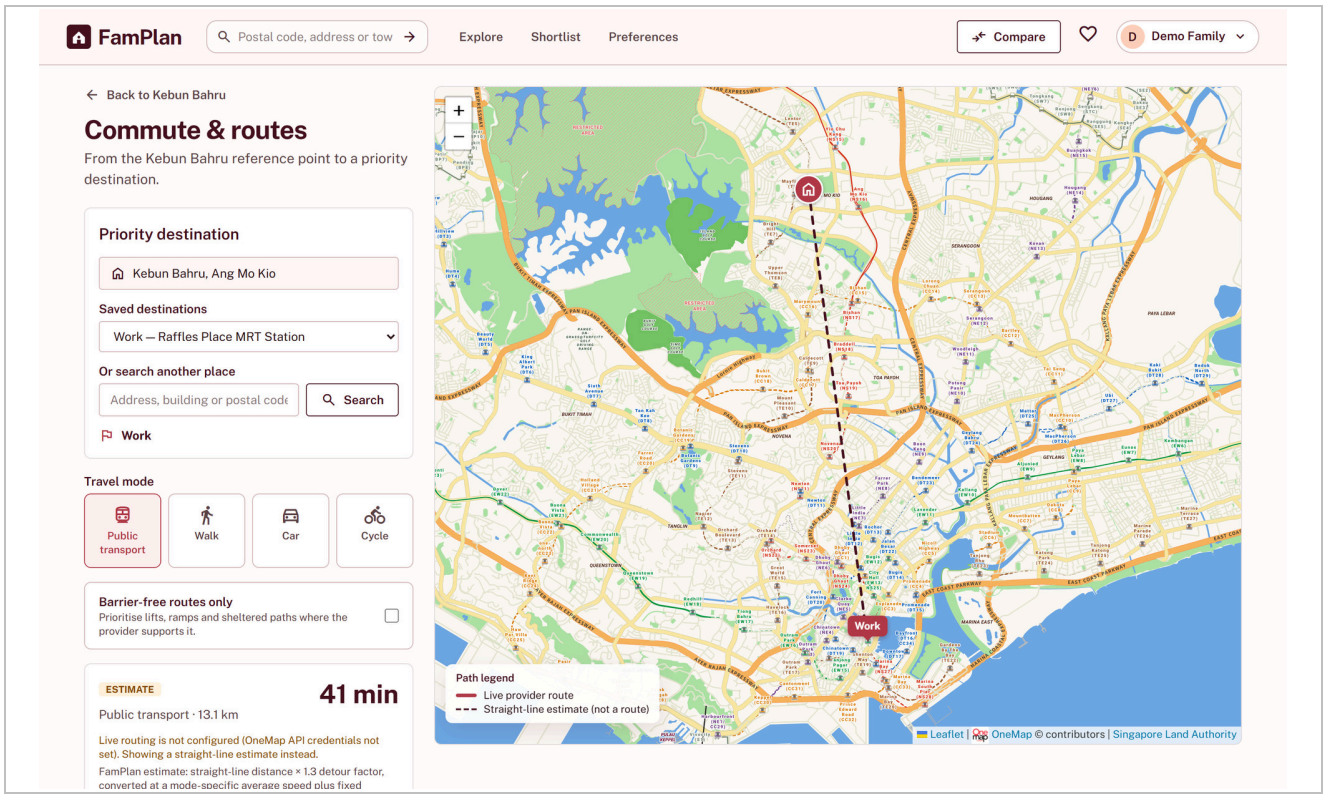


Figure 3.8. Commute and routes (UC-5.2). Saved or searched destination, travel mode, barrier-free option, and the route estimate with its provider. This local build has no OneMap credentials, so it shows the labelled straight-line estimate.

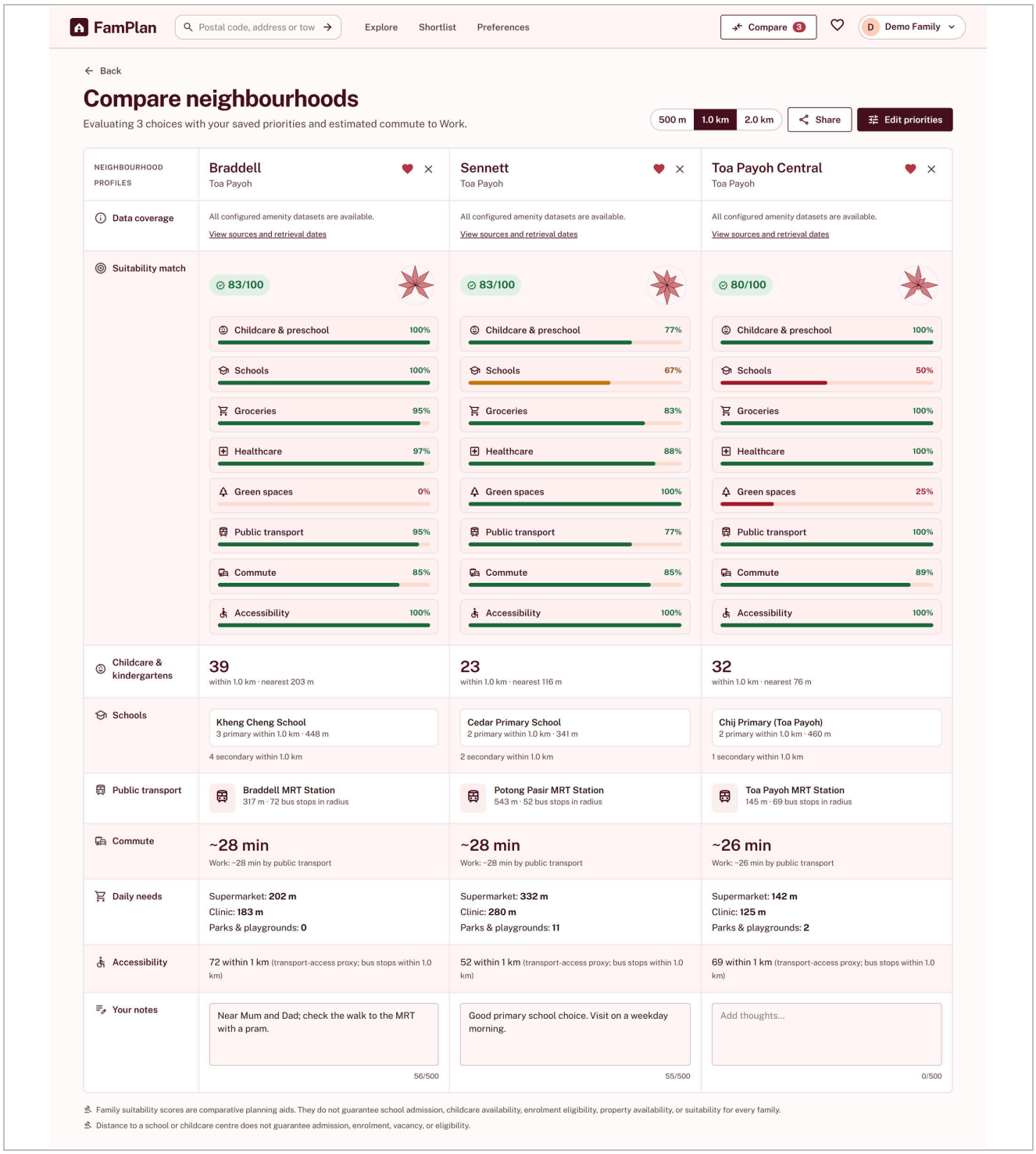


Figure 3.9. Compare neighbourhoods (UC-6.2, UC-6.1). Two to four neighbourhoods side by side with the same weights and radius: score breakdown, amenities, transport, commute, daily needs and accessibility.

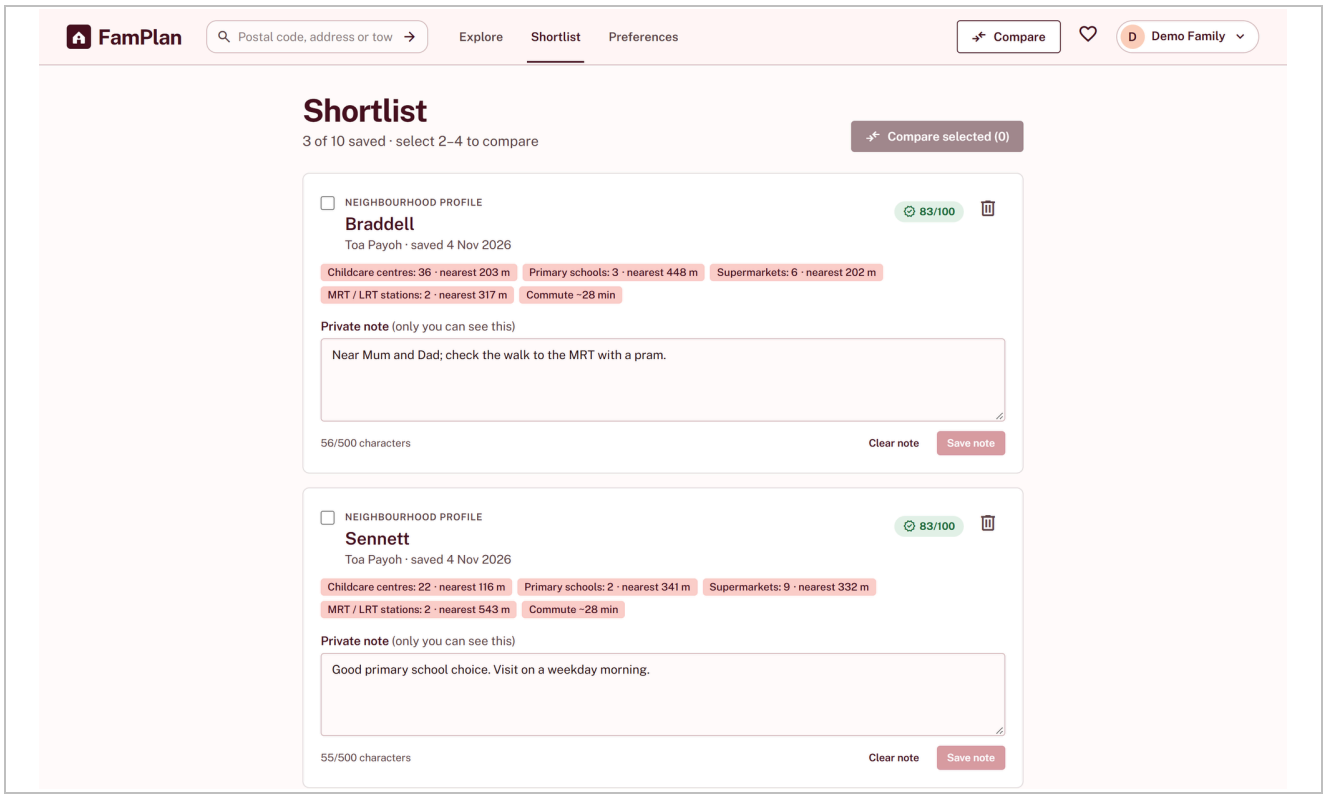


Figure 3.10. Shortlist and private notes (UC-6.3). Saved neighbourhoods (up to ten) with private notes of up to 500 characters and selection for comparison.

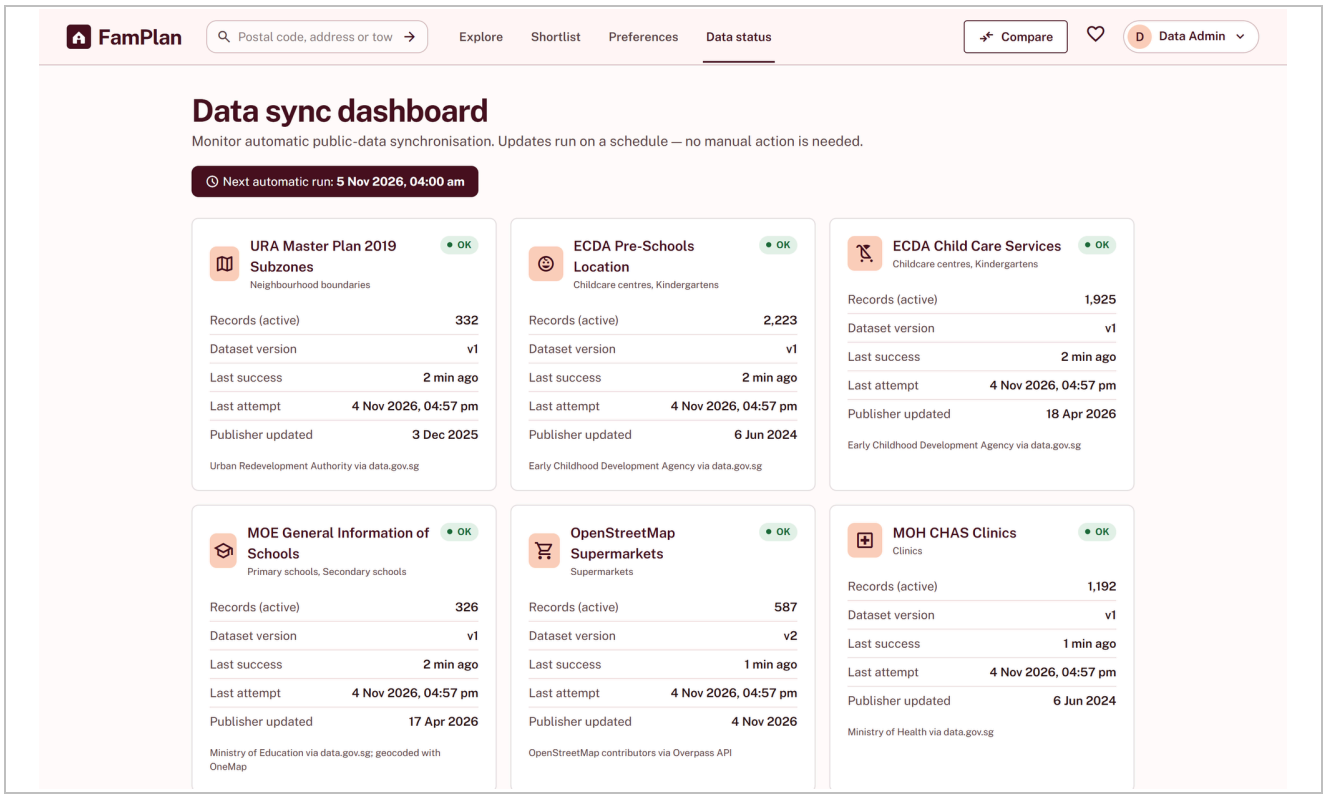


Figure 3.11. Data sync dashboard (UC-7.1). Administrator-only view of each source's status, record count, dataset version and latest successful and attempted synchronisation.

3.2 Hardware Interfaces

No specialised hardware is required. Users interact through a browser using a keyboard, pointer, or touch input. The application does not require device sensors or a native mobile application.

3.3 Software Interfaces

Interface	Planned Information	Use and Failure Behaviour
MOE or data.gov.sg education datasets	Primary and secondary school names, types, addresses, and published metadata	Supports school proximity; does not predict admission.
ECDA or authorised early-childhood dataset	Childcare and kindergarten locations and categories	Supports early-childhood proximity; does not claim live vacancy.
Authorised retail and healthcare datasets	Supermarket and clinic locations	Supports essential-service proximity with category-level unavailable states.
OneMap	Geocoding, map display, routing, and supported place themes	Supports maps and commute functions; outage does not disable locally cached neighbourhood data.
LTA DataMall	Public-transport stop or station information	Supplements transport accessibility evidence.
Email Delivery Service	Password-reset message delivery	Supports UC-1.3 and must not reveal whether an address is registered.

3.4 Communications Interfaces

Client-server and external-service traffic uses HTTPS with TLS 1.2 or newer. Requests use validated parameters and provider-specific timeouts and rate limits. Automatic data synchronisation is initiated by the server schedule at least every 24 hours; administrators do not manually trigger it. Password reset is the only email interface in scope.

4. Functional Requirements

The following 49 atomic functional requirements define the family-neighbourhood product. Each requirement identifies an actor, input, required output, and minimum verification evidence.

4.1 Account Management

Create and manage accounts, authentication, display names, and account deletion.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-ACC-01 Must Visitor	The system shall create a user account after receiving valid registration details and consent to the stated data use.	Input: Display name, email, password, consent Output: Account confirmation or field-specific errors	Submit valid, duplicate-email, weak-password, malformed-email, and missing-consent cases.
FR-ACC-02 Must Registered User	The system shall authenticate a Registered User with valid credentials.	Input: Email and password Output: Authenticated session or generic failure	Test valid, invalid, unknown, locked, and expired-session cases without revealing account existence.
FR-ACC-03 Must Registered User	The system shall end the current authenticated session when the user logs out.	Input: Logout action Output: Invalidated session and public-page redirect	Log out and verify that the previous session token cannot access a protected function.
FR-ACC-04 Should Registered User	The system shall support password reset through a time-limited, single-use link sent to the registered email address.	Input: Email address and new password Output: Generic confirmation and completed reset or expiry error	Test registered and unknown emails, expired and reused links, and password-policy boundaries.
FR-ACC-05 Must Registered User	The system shall allow a Registered User to update the display name stored on the account.	Input: New display name Output: Updated display name or validation error	Update the display name and verify that email and credentials cannot be changed through this function.
FR-ACC-06 Must Registered User	The system shall delete the user's account, saved preferences, shortlist, and private notes after explicit confirmation.	Input: Confirmed deletion request Output: Deletion acknowledgement and ended session	Confirm and cancel deletion, then verify that removed data is no longer retrievable within the stated period.

4.2 Family and Neighbourhood Preferences

Store family stages, preferred areas, destinations, amenity thresholds, and criterion weights.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-PREF-01 Must Registered User	The system shall store the household's selected family-planning stages, including expecting a child, infant care, preschool, primary school, and secondary school.	Input: One or more family stages Output: Saved family-stage preferences	Save every supported stage alone and in combination; reject unknown values.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-PREF-02 Must Registered User	The system shall store preferred towns or planning areas for neighbourhood discovery.	Input: Selected towns or planning areas Output: Saved area preferences	Save, remove, duplicate, and unknown area selections.
FR-PREF-03 Must Registered User	The system shall store up to three priority destinations, each with a travel mode, and shall use all of them for commute scoring and comparison.	Input: Destination labels, addresses and travel modes Output: Saved and geocoded destinations or field errors	Test zero through four destinations, duplicates, unresolved addresses, and every travel mode.
FR-PREF-04 Must Registered User	The system shall store selected family amenity categories and a preferred maximum distance for each selected category.	Input: Amenity categories and distance thresholds Output: Saved amenity preferences	Test all supported categories and distances below, on, and above allowed boundaries.
FR-PREF-05 Must Registered User	The system shall store an integer importance weight from 0 to 5 for childcare, schools, groceries, healthcare, green spaces, public transport, commute, and accessibility.	Input: Eight criterion weights Output: Saved weights or field errors	Test -1, 0, 5, 6, non-integer, partial, and all-zero weight sets.
FR-PREF-06 Must Registered User	The system shall retrieve and update the user's saved family and neighbourhood preferences.	Input: Authenticated preference request Output: Current or updated preference profile	Save preferences, sign out, sign in, retrieve them, update them, and confirm ownership isolation.
FR-PREF-07 Should Registered User	The system shall offer editable starting weights based on the selected family-planning stages without overwriting choices already confirmed by the user.	Input: Family stages and preset action Output: Editable suggested weights	Apply each preset, edit it, and verify that a later stage change does not overwrite confirmed values without consent.

4.3 Neighbourhood Discovery

Search, filter, label, sort, and validate neighbourhood profiles.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-DISC-01 Must Visitor	The system shall search neighbourhoods by town, planning area, estate name, or address.	Input: Search text or selected area Output: Matching neighbourhood profiles	Test known and unknown towns, areas, estates, addresses, spelling variants, and empty search.
FR-DISC-02 Must Visitor	The system shall filter neighbourhood results by family stage, amenity category, minimum facility count, maximum nearest-facility distance, and commute-time threshold.	Input: Filter values Output: Filtered neighbourhood profiles	Test each filter independently, in valid combinations, and at every numeric boundary.
FR-DISC-03 Must Registered User	The system shall apply saved family and neighbourhood preferences to the current search.	Input: Apply-preferences action Output: Populated filters and refreshed results	Test complete, partial, empty, and outdated preference profiles without silently changing saved values.
FR-DISC-04 Must Visitor	The system shall identify every result as a neighbourhood profile rather than a live property listing or guarantee of service availability.	Input: Search results Output: Visible neighbourhood-profile and limitation labels	Inspect every result layout, empty state, comparison, shortlist, and shared view.
FR-DISC-05 Must Visitor	The system shall sort neighbourhood results by family suitability score, nearest selected amenity, commute time, or overall amenity coverage when the selected value is available.	Input: Sort choice Output: Ordered results with unavailable values placed last	Verify ascending and descending order, ties, missing values, and stable pagination.
FR-DISC-06 Must Visitor	The system shall reject invalid or contradictory discovery criteria before executing a search.	Input: Submitted search and filter criteria Output: Field-specific correction guidance	Test negative counts, unsupported distances, invalid destinations, contradictory thresholds, and unexpected types.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-DISC-07 Must Visitor	The system shall resolve a postal code, street address or place name to one exact location, ask the user to choose when several places match, and calculate the radius, counts, score evidence and routes from that location's coordinate.	Input: Postal code, address or place name, and a chosen match when several exist Output: Results based on the exact location, a list of matches, or a no-match message	Test leading-zero postal codes, ambiguous places, no match, provider failure, and radius boundaries from the selected coordinate.

4.4 Family Amenities and Neighbourhood Information

Present family-relevant facilities, proximity measures, counts, sources, and limitations.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-NBH-01 Must Visitor	The system shall display a selected neighbourhood's name, town or planning area, map location, and the boundary or coverage basis used for its analysis.	Input: Selected neighbourhood Output: Neighbourhood summary and coverage explanation	Open a known profile and verify all fields, source metadata, and missing-boundary handling.
FR-NBH-02 Must Visitor	The system shall retrieve nearby childcare centres, kindergartens, primary schools, secondary schools, supermarkets, clinics, parks or playgrounds, MRT stations, and bus stops for a selected neighbourhood.	Input: Neighbourhood and amenity category Output: Categorised nearby facilities or category-specific unavailable state	Use fixtures containing every category, no matches, duplicate records, and a failed provider.
FR-NBH-03 Must Visitor	The system shall allow nearby-facility results to be limited to a 500 m, 1 km, or 2 km radius.	Input: Selected radius Output: Facilities within the radius	Test places immediately inside, exactly on, and immediately outside every radius.
FR-NBH-04 Must Visitor	The system shall display the calculated distance from the neighbourhood reference point to each nearby facility and state the calculation basis.	Input: Neighbourhood and facility coordinates Output: Distance, unit, and basis	Compare displayed values with known coordinate fixtures and documented rounding.
FR-NBH-05 Must Visitor	The system shall display the number of matched facilities in each supported family amenity category for the selected radius.	Input: Categorised nearby-facility results Output: Per-category counts	Verify zero, one, many, boundary, and duplicate-source cases.
FR-NBH-06 Must Visitor	The system shall identify the nearest matched facility in each selected category when at least one match is available.	Input: Categorised facility distances Output: Nearest facility and distance or no-match state	Test ties, no matches, missing coordinates, and deterministic ordering.
FR-NBH-07 Must Visitor	The system shall display each facility's name, category, address where available, and source.	Input: Selected facility Output: Facility details with explicit unavailable fields	Inspect every category with complete and incomplete source records.
FR-NBH-08 Must Visitor	The system shall display the provider and applicable update or retrieval time for neighbourhood and facility information.	Input: External response metadata Output: Visible provenance information	Inspect fresh, cached, merged, and unavailable results.
FR-NBH-09 Must Visitor	The system shall distinguish no matched facility, incomplete category coverage, stale cached information, and source unavailability instead of presenting a misleading zero.	Input: Data availability and freshness state Output: Explicit category-level status	Simulate each state and verify that unaffected categories remain visible.
FR-NBH-10 Must Visitor	The system shall state that distance to a school or childcare centre does not guarantee admission, enrolment, vacancy, or eligibility.	Input: Education or childcare output Output: Visible limitation statement	Inspect neighbourhood, comparison, shortlist, and shared result views.

4.5 Map and Travel

Show neighbourhood maps, nearby places, routes, travel times, and supported accessibility coverage.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-TRV-01 Must Visitor	The system shall display the selected neighbourhood and matched facilities on an interactive map with distinguishable category markers.	Input: Neighbourhood and nearby facilities Output: Interactive categorised map	Test marker categories, keyboard access, overlapping points, and missing coordinates.
FR-TRV-02 Must Visitor	The system shall request a route and estimated travel time from the selected neighbourhood to a chosen priority destination.	Input: Origin, destination, and travel mode Output: Route, distance, duration, or unavailable state	Test valid, invalid, timeout, no-route, and unresolved-destination cases.
FR-TRV-03 Must Visitor	The system shall support the travel modes made available by the authorised routing provider and clearly reject unsupported modes.	Input: Travel-mode selection Output: Supported route or explanatory rejection	Test every configured and unsupported mode against controlled responses.
FR-TRV-04 Should Visitor	When barrier-free routing is requested, the system shall show a barrier-free route where the authorised provider supplies one or state that coverage is unavailable.	Input: Origin, destination, barrier-free option Output: Accessible route or coverage limitation	Test supported, unsupported, and partially covered route fixtures.
FR-TRV-05 Must Visitor	The system shall display the provider and retrieval time for route information.	Input: Routing response metadata Output: Visible provider and retrieval time	Inspect successful, cached, and unavailable route states.

4.6 Family Decision Support

Calculate transparent family suitability scores, compare neighbourhoods, and manage a shortlist.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-DEC-01 Must Visitor	The system shall calculate a family suitability score for each neighbourhood using the selected criterion weights and disclosed normalisation rules.	Input: Neighbourhood evidence and criterion weights Output: Total score on a 0 to 100 scale	Compare representative and boundary results with independently calculated fixtures.
FR-DEC-02 Must Visitor	The system shall display each criterion's value, normalised value, weight, and contribution so the contributions reconcile with a total rounded to the nearest whole number, with an exact 0.5 rounded upward.	Input: Score calculation Output: Itemised explanation and total	Recalculate typical, boundary, all-zero, and exact-0.5 cases.
FR-DEC-03 Must Visitor	The system shall identify every criterion lacking sufficient data and disclose how its weight was treated in the score.	Input: Neighbourhood with missing criterion data Output: Missing-data label and adjusted-weight explanation	Remove each criterion in turn and verify the disclosed rule and resulting total.
FR-DEC-04 Must Visitor	The system shall compare between two and four selected neighbourhoods in a common view.	Input: Two to four neighbourhood selections Output: Side-by-side family amenity, commute, accessibility, and score information	Test zero through five selections, duplicates, removal, and missing values.
FR-DEC-05 Must Registered User	The system shall add a selected neighbourhood to the Registered User's shortlist, up to a maximum of ten neighbourhoods, without creating a duplicate entry.	Input: Add-to-shortlist action Output: Persisted entry, already-saved state, or capacity explanation	Add through the tenth entry, attempt an eleventh, add a duplicate, and verify persistence after sign-in.
FR-DEC-06 Must Registered User	The system shall remove a selected neighbourhood from the user's shortlist.	Input: Remove action Output: Updated shortlist and confirmation	Remove existing and non-existing entries and verify persistent state.
FR-DEC-07 Should Registered User	The system shall allow a private text note of up to 500 characters to be created, edited, and cleared for each shortlisted neighbourhood.	Input: Shortlisted neighbourhood and note text Output: Persisted note or validation error	Test create, update, clear, 499-, 500-, and 501-character values, unsafe markup, and cross-account access.

4.7 Automatic Data Synchronisation and Resilience

Automatically update configured public data, let administrators monitor outcomes, and preserve validated data after failures.

ID Priority Actor	Atomic Requirement	Input and Required Output	Verification
FR-ADM-01 Must Data Administrator	The system shall require an authenticated administrator role before displaying data-source monitoring functions.	Input: Authenticated session and monitoring request Output: Monitoring page or access denial	Test anonymous, ordinary-user, expired-admin, and active-admin sessions at UI and API levels.
FR-ADM-02 Must Data Administrator	The system shall display each configured source's latest successful synchronisation time, latest attempted synchronisation time, record count, and current status.	Input: Administration-page request Output: Source-status summary	Compare the view with controlled successful, failed, never-run, and delayed states.
FR-ADM-03 Must System	The system shall automatically check and synchronise each configured public data source at least once every 24 hours without requiring an administrator to start the process.	Input: Configured schedule and source metadata Output: Recorded check, synchronisation outcome, and active dataset decision	Advance a test clock across the boundary and verify that no manual action is required.
FR-ADM-04 Must Data Administrator	The system shall display synchronisation history containing source, start time, end time, outcome, records added or changed, and a sanitised error summary.	Input: Source and optional date filter Output: Reverse-chronological synchronisation history	Generate successes and failures and verify fields, ordering, filtering, and absence of secrets.
FR-RES-01 Must System	The system shall preserve the last validated local dataset when an automatic synchronisation fails validation or cannot complete.	Input: Failed synchronisation Output: Previous active dataset and recorded failure	Query before and after injected download, schema, and validation failures.
FR-RES-02 Must System	The system shall isolate an external map, route, or facility-service failure so account, shortlist, and locally cached neighbourhood functions remain usable.	Input: Simulated dependency failure Output: Controlled unavailable state for affected features	Disable each dependency and execute every named unaffected workflow.
FR-RES-03 Must Visitor	The system shall show the age of cached external information and a stale-data warning when the configured freshness threshold is exceeded.	Input: Cached-result metadata Output: Data age and applicable stale warning	Test values immediately below, on, and above the configured threshold.

5. Nonfunctional Requirements

The following 29 measurable requirements constrain performance, scalability, usability, accessibility, compatibility, reliability, data quality, security, privacy, maintainability, testability, and transparency.

5.1 Performance Requirements

ID and Priority	Measurable Requirement	Verification
NFR-PERF-01 Must	The 95th-percentile server response time for neighbourhood search and filtering must not exceed 2 seconds during a 15-minute test with 100 concurrent simulated users against at least 100,000 facility and neighbourhood records, excluding live external API calls.	Load-test report containing dataset size, concurrency, request count, errors, and percentile timings.
NFR-PERF-02 Must	The 95th-percentile response time for locally calculated family scores and neighbourhood comparisons must not exceed 3 seconds under the NFR-PERF-01 workload.	Load-test representative two-, three-, and four-neighbourhood comparisons.

ID and Priority	Measurable Requirement	Verification
NFR-PERF-03 Should	The 95th-percentile end-to-end response time for a live route request must not exceed 5 seconds while the provider is within documented normal operating conditions.	Timed integration tests that distinguish provider time from local processing and include controlled timeouts.

5.2 Scalability Requirements

ID and Priority	Measurable Requirement	Verification
NFR-SCALE-01 Must	The deployed application must store, index, and query at least 100,000 neighbourhood, facility, transport, and provenance records without violating NFR-PERF-01 or NFR-PERF-02.	Seed the required volume, record indexes, and execute the performance tests.
NFR-SCALE-02 Must	The deployed application must support at least 100 concurrent active users performing representative search, profile, map, score, comparison, shortlist, and cached-route workflows without violating applicable performance targets.	Mixed-workload load test with response and error statistics by operation.

5.3 Usability Requirements

ID and Priority	Measurable Requirement	Verification
NFR-USAB-01 Should	At least 80% of first-time representative users in a moderated test of at least 10 participants must search for a neighbourhood, select two results, and open their comparison within 3 minutes without facilitator assistance.	Anonymised usability protocol, observations, timings, completion rate, and issues.
NFR-USAB-02 Must	Every rejected user input must identify the affected field or action and state how the user can correct it; a generic error code alone is insufficient.	UI and API acceptance tests covering every documented validation error.

5.4 Accessibility Requirements

ID and Priority	Measurable Requirement	Verification
NFR-ACC-01 Must	The account, preference, search, neighbourhood, map, comparison, shortlist, and administration workflows must conform to WCAG 2.1 Level AA.	Automated accessibility scan plus manual checks for labels, headings, contrast, errors, zoom, maps, and screen-reader output.
NFR-ACC-02 Must	Every interactive function in the core workflows must be operable using only a keyboard and must expose a visible focus indicator.	Manual keyboard traversal using a documented checklist with no pointer input.

5.5 Compatibility Requirements

ID and Priority	Measurable Requirement	Verification
NFR-COMP-01 Must	Core pages must remain usable from 360 px to 1440 px viewport widths without horizontal scrolling of the primary page content.	Screenshot or browser-test evidence at boundary and intermediate widths.
NFR-COMP-02 Should	Core workflows must operate on the latest two stable versions available at test time of Chrome, Safari, Firefox, and Edge.	Compatibility matrix recording browser versions and pass or fail results.

5.6 Reliability Requirements

ID and Priority	Measurable Requirement	Verification
NFR-REL-01 Must	At least 99% of core requests must complete successfully during a continuous one-hour representative-load test, excluding requests whose sole failure is an injected or confirmed external-provider outage.	One-hour report with request totals, failure classification, and evidence for exclusions.

5.7 Recoverability Requirements

ID and Priority	Measurable Requirement	Verification
NFR-REL-02 Must	Following an application restart, locally stored account, preference, search, profile, shortlist, administration, and cached-neighbourhood functions must become operational within 5 minutes without manual reconstruction of persistent data.	Timed restart-and-recovery exercise followed by integrity and smoke tests.

5.8 Fault isolation Requirements

ID and Priority	Measurable Requirement	Verification
NFR-REL-03 Must	Failure of one external dependency must not prevent use of unrelated functions backed by validated local data.	Dependency-failure matrix showing affected and unaffected workflows.

5.9 Data provenance Requirements

ID and Priority	Measurable Requirement	Verification
NFR-DATA-01 Must	Every neighbourhood, facility, transport, route, and family-score output must display or link to its source and applicable update or retrieval date.	Automated presence checks plus manual inspection of every output and error state.

5.10 Data freshness Requirements

ID and Priority	Measurable Requirement	Verification
NFR-DATA-02 Must	Locally cached neighbourhood and facility data older than 31 complete days must be visibly marked as stale until a newer validated dataset becomes active.	Set test-data ages to 30, 31, and 32 complete days and verify boundary behaviour.

5.11 Reproducibility Requirements

ID and Priority	Measurable Requirement	Verification
NFR-DATA-03 Must	Identical neighbourhood inputs, user weights, scoring-configuration version, and dataset versions must produce identical family scores and comparison results.	Repeat deterministic fixture tests across process restarts.

5.12 Data quality Requirements

ID and Priority	Measurable Requirement	Verification
NFR-DATA-04 Must	Facility records merged from multiple sources must be normalised by supported category and must not be counted twice when they represent the same named facility at the same location.	Use exact, near-duplicate, renamed, and distinct co-located facility fixtures and inspect counts.

5.13 Transport security Requirements

ID and Priority	Measurable Requirement	Verification
NFR-SEC-01 Must	Every production client-server connection must use HTTPS with TLS 1.2 or newer, and plain HTTP must redirect to HTTPS without accepting credentials.	Protocol scan, redirect test, and attempted authentication over plain HTTP.

5.14 Credential security Requirements

ID and Priority	Measurable Requirement	Verification
NFR-SEC-02 Must	Passwords must be stored only as salted hashes using a current approved password-hashing algorithm and must never appear in application logs.	Inspect schema and configuration; scan logs after registration, login, failure, and reset tests.

5.15 Authorisation Requirements

ID and Priority	Measurable Requirement	Verification
NFR-SEC-03 Must	A non-administrator must be unable to read data-source monitoring functions even when directly calling their endpoints.	Role-based access-control tests at both UI and API levels.

5.16 Authentication security Requirements

ID and Priority	Measurable Requirement	Verification
NFR-SEC-04 Must	An account must be locked from password authentication for 15 minutes after five consecutive failed login attempts; a successful login before the threshold must reset the counter.	Test attempts 1-6, successful reset, and 14-, 15-, and 16-minute boundaries.

5.17 Input security Requirements

ID and Priority	Measurable Requirement	Verification
NFR-SEC-05 Must	All user-controlled input must be allow-list or structurally validated before use in a query, calculation, stored note, map request, or external-service request.	Security tests for injection, unsafe markup, malformed coordinates, invalid ranges, and unexpected types.

5.18 Privacy Requirements

ID and Priority	Measurable Requirement	Verification
NFR-PRIV-01 Must	Account details, saved family preferences, shortlists, and notes must become irretrievable through the application within 24 hours after confirmed account deletion.	Delete a seeded account and test UI, API, and authorised data-store lookup at the deadline.

5.19 Data minimisation Requirements

ID and Priority	Measurable Requirement	Verification
NFR-PRIV-02 Must	Registration and family preferences must not require exact dates of birth, children's names, school applications, medical information, or other personal information unnecessary for the documented functions.	Compare required fields with the approved data dictionary and submit all workflows without excluded fields.

5.20 Maintainability Requirements

ID and Priority	Measurable Requirement	Verification
NFR-MAINT-01 Must	Data-source endpoints, freshness thresholds, amenity categories, scoring criteria, and normalisation constants must be maintained as versioned configuration or isolated modules so one category can be updated without changing unrelated workflow code.	Code review plus tests that replace one configuration category while unaffected workflows still pass.

5.21 Testability Requirements

ID and Priority	Measurable Requirement	Verification
NFR-MAINT-02 Should	Automated tests must achieve at least 80% branch coverage for family scoring, proximity aggregation, facility deduplication, and external-data transformation modules.	Coverage report generated by the project's documented test command.

5.22 Transparency Requirements

ID and Priority	Measurable Requirement	Verification
NFR-TRANS-01 Must	Every family suitability score must expose criterion values, weights, contributions, dataset versions, and missing-data treatment needed to reproduce it subject to documented rounding.	Recalculate representative outputs independently from the displayed explanation.

5.23 Responsible guidance Requirements

ID and Priority	Measurable Requirement	Verification
NFR-TRANS-02 Must	The user interface must state that neighbourhood scores are comparative planning aids and do not guarantee school admission, childcare availability, enrolment eligibility, property availability, or suitability for every family.	Review every score, comparison, shortlist, and shareable output for the limitation statement.

5.24 Business Rules

Rule	Value	Requirements
Neighbourhood comparison	2 to 4 neighbourhoods	FR-DEC-04
Shortlist capacity	Maximum 10 neighbourhoods per Registered User	FR-DEC-05
Private note length	0 to 500 characters	FR-DEC-07
Proximity radii	500 m, 1 km, or 2 km	FR-NBH-03
Criterion weights	Integer 0 to 5	FR-PREF-05
Score rounding	Nearest whole number; exact 0.5 rounds upward	FR-DEC-02
Automatic synchronisation	At least once every 24 hours; system initiated	FR-ADM-03
Editable account detail	Display name	FR-ACC-05
Commute time	Average across all destinations, each by its travel mode	FR-PREF-03; FR-DEC-01

6. Data Dictionary

The table defines the shared terms used by the requirements, use cases, implementation, and future tests. Examples are illustrative; the attributes and constraints are the baseline.

Term	Definition	Key Attributes	Relationships and Constraints
User Account	Registered person's identity in FamPlan.	id; display_name; email; consent_given_at; status	Owns one preference profile and zero or more shortlist entries; display name is the editable general account field.
Password Credential	Stored proof of a password, never the password itself.	account_id; salted_hash; algorithm; parameters	Belongs to one User Account; never logged.
Session	Current authenticated login.	token; account_id; created_at; expires_at; invalidated_at	Belongs to one User Account; invalidated immediately on logout.
Password Reset Token	Time-limited, single-use credential for account recovery.	token; account_id; expires_at; used_at	Belongs to one User Account; delivery must not reveal whether an email exists.
Administrator Role	Privilege permitting data-source monitoring.	account_id; granted_at	At most one per User Account; enforced at UI and API levels.

Term	Definition	Key Attributes	Relationships and Constraints
Family Preference Profile	Saved settings used to personalise neighbourhood discovery.	account_id; family_stages; preferred_areas; amenity_thresholds; criterion_weights	At most one per Registered User; applied to a Search Query.
Family Stage	Planning stage relevant to nearby services.	enum: expecting_child; infant_care; preschool; primary_school; secondary_school	One or more values may be selected; no names or dates of birth are required.
Amenity Category	Supported type of family-relevant place.	childcare; kindergarten; primary_school; secondary_school; supermarket; clinic; park_playground; mrt; bus_stop	Classifies Facility records and links preferences to score criteria.
Criterion Weight	User importance assigned to a scoring criterion.	criterion; integer weight 0 to 5	Part of a Family Preference Profile; all-zero input uses the documented fallback.
Search Query	Request-scoped neighbourhood search and filters.	search_text; area; stages; categories; counts; distances; commute threshold; sort	May be populated from saved preferences; invalid combinations are rejected before execution.
Neighbourhood Profile	Comparable representation of a town, planning area, estate, or documented coverage zone.	id; name; town; planning_area; reference_point; boundary_basis	Has many Facility Matches, routes, score breakdowns, and shortlist references; it is not a property listing.
Geolocation	Latitude and longitude pair.	latitude; longitude; source	Attached to neighbourhoods, facilities, and destinations.
Selected Location	Exact address chosen as the origin of a search.	address; latitude; longitude; postal_code	Request-scoped. Replaces the neighbourhood reference point for radius, counts, score evidence and routes.
Facility	Childcare, education, retail, healthcare, recreation, or transport place.	id; name; category; address; geolocation; source_id; source_updated_at	May appear in many neighbourhood-radius results; duplicates are resolved by the documented matching rule.
Proximity Radius	Boundary used when retrieving nearby facilities.	metres: 500; 1000; 2000	Bounds Facility Matches for one neighbourhood query.
Facility Match	Facility located within a selected radius of a neighbourhood reference point.	neighbourhood_id; facility_id; distance_m; distance_basis	Joins one Neighbourhood Profile to one Facility for a specific calculation.
Amenity Summary	Per-category neighbourhood aggregation.	category; matched_count; nearest_facility_id; nearest_distance_m; availability_state	Computed from Facility Matches and used by scoring and comparison.
Priority Destination	Saved place and travel mode used for commute scoring and comparison.	slot 1 to 3; label; query_text; geolocation; travel_mode	Part of a Family Preference Profile and destination of a Route Estimate.
Route Estimate	Provider response for travel from a neighbourhood to a destination.	origin; destination; mode; distance_m; duration_s; retrieved_at; state	Carries provenance and may include a barrier-free coverage flag.
Scoring Configuration	Versioned definitions and normalisation constants for family suitability.	version; effective_date; criteria; normalisation rules	Referenced by every Family Suitability Score.
Family Suitability Score	0-100 indication of how well a neighbourhood matches selected priorities.	neighbourhood_id; configuration_version; dataset_versions; total; computed_at	Contains Criterion Score Breakdowns; comparative planning aid, not a guarantee.
Criterion Score Breakdown	One criterion's transparent contribution to a total score.	criterion; raw_value; normalised_value; weight; contribution	Part of one Family Suitability Score.
Missing Data Treatment	Explanation of how unavailable evidence affects scoring.	criterion; reason; weight_treatment	Attached to a score breakdown when a value cannot be computed.
Neighbourhood Comparison Set	Transient selection of two to four neighbourhoods.	neighbourhood_ids	Request-scoped; contains no duplicates.

Term	Definition	Key Attributes	Relationships and Constraints
Shortlist Entry	Saved neighbourhood selected by a Registered User.	account_id; neighbourhood_id; added_at	Maximum ten per account; unique by account and neighbourhood; may have one note.
Shortlist Note	Private text attached to a shortlisted neighbourhood.	shortlist_entry_id; text; updated_at	Maximum 500 characters; readable only by the owning account.
Data Source Registry	Configuration and current health of each public data source.	source_id; name; schedule; freshness_threshold; last_attempt; last_success; status	Has many Sync History Records and Validated Dataset Snapshots.
Sync History Record	Outcome of one automatic API update attempt.	source_id; start; end; outcome; records_added; records_changed; sanitised_error	Created automatically; displayed to the Data Administrator; contains no manual-trigger field.
Sync Lease	Cross-instance lock for one data source's refresh.	source_id; owner; expires_at	Prevents duplicate runs. An expired lease can be taken over after an interrupted refresh.
Validated Dataset Snapshot	Source version that passed validation and can be served.	source_id; version; validated_at; active	Only a validated snapshot becomes active; prior active data remains after failure.
Data Provenance Stamp	Source and applicable data date shown with externally derived output.	provider; source_updated_at; retrieved_at; dataset_version	Attached to neighbourhood, facility, route, and score evidence.

7. Use Case Model

7.1 Use Case Diagram

Figure 7.1 shows the 16 use cases inside the FamPlan system boundary, grouped as in Section 4. Registered User specialises Visitor and Data Administrator specialises Registered User, so inherited associations are not redrawn. UC-6.1 has no initiating actor because it is included by UC-4.1 and UC-6.2; UC-3.2 extends UC-3.1. Scheduled synchronisation (FR-ADM-03) is supporting system behaviour, so the administrator only monitors it (UC-7.1).

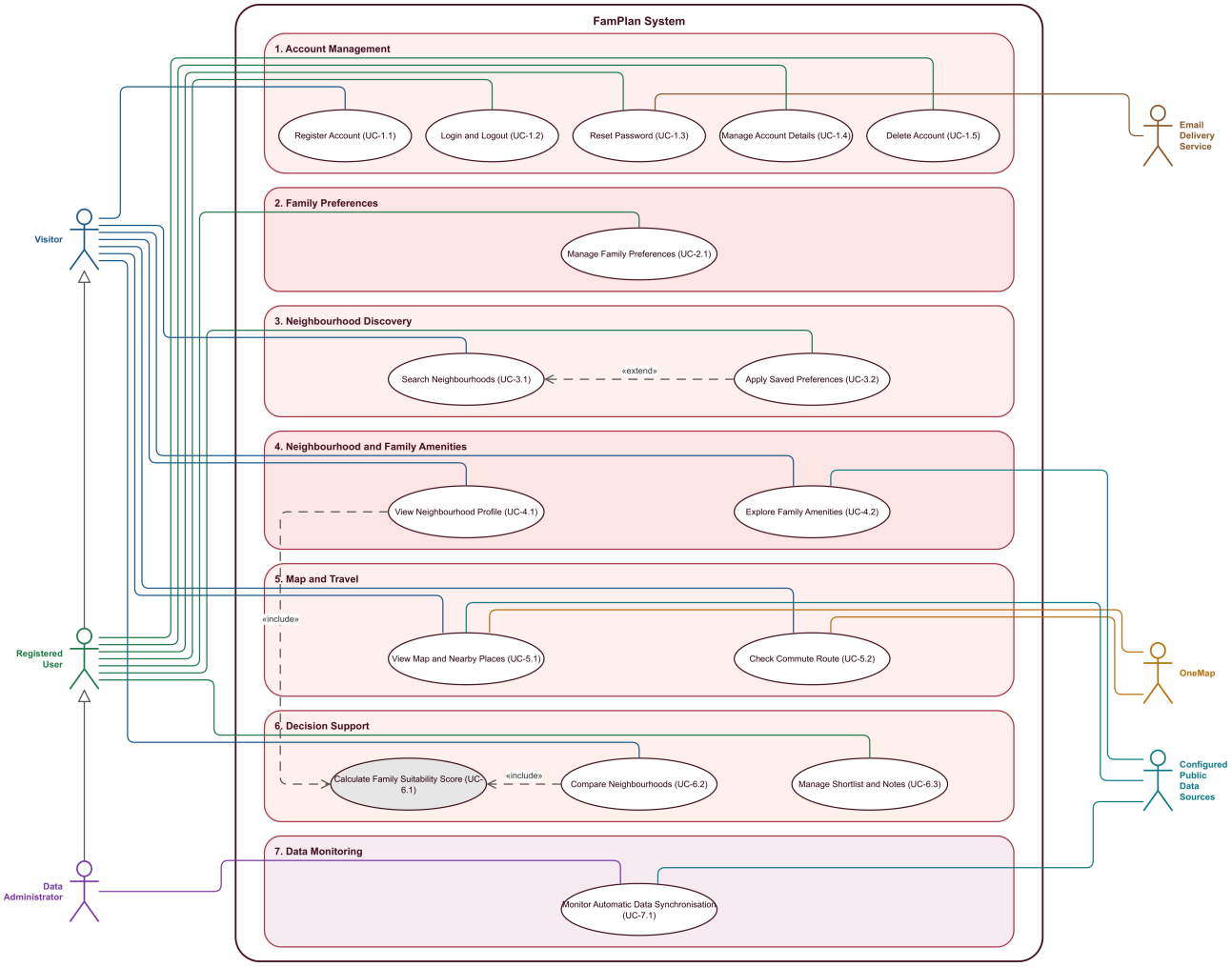


Figure 7.1. FamPlan use case diagram.

7.2 Actors

Actor	Type	Description
Visitor	Primary	Any member of the public using FamPlan without signing in.
Registered User	Primary	A signed-in household member; inherits every Visitor use case.
Data Administrator	Primary	A Registered User with the DATA_ADMINISTRATOR role; monitors automatic synchronisation only.
Email Delivery Service	Supporting	Sends password-reset emails (UC-1.3).
OneMap	Supporting	Singapore Land Authority map, address search and routing service (UC-5.1, UC-5.2).
Configured Public Data Sources	Supporting	data.gov.sg datasets (URA, ECDA, MOE, MOH, NParks, LTA) and OpenStreetMap, synchronised automatically (UC-4.2, UC-5.1, UC-7.1).

7.3 Use Case Catalogue

ID	Use Case	Actor or Supporting System	Goal	Requirements	Relationship
UC-1.1	Register Account	Visitor	Create a personal account.	FR-ACC-01	None
UC-1.2	Login and Logout	Registered User	Start or end an authenticated session.	FR-ACC-02 to FR-ACC-03	None
UC-1.3	Reset Password	Registered User; Email Delivery Service	Recover access through a secure reset link.	FR-ACC-04	None
UC-1.4	Manage Account Details	Registered User	Update the account display name.	FR-ACC-05	None
UC-1.5	Delete Account	Registered User	Confirm deletion of the account and saved data.	FR-ACC-06	None
UC-2.1	Manage Family Preferences	Registered User	Save family stages, areas, amenities, destinations, and weights.	FR-PREF-01 to FR-PREF-07	None
UC-3.1	Search Neighbourhoods	Visitor	Find and filter neighbourhood profiles.	FR-DISC-01 to FR-DISC-02; FR-DISC-04 to FR-DISC-07	Base use case
UC-3.2	Apply Saved Preferences	Registered User	Populate search criteria from saved preferences.	FR-DISC-03	Extends UC-3.1
UC-4.1	View Neighbourhood Profile	Visitor	Inspect the selected neighbourhood and its family score.	FR-NBH-01; FR-DEC-01 to FR-DEC-03	Includes UC-6.1
UC-4.2	Explore Family Amenities	Visitor; Configured Public Data Sources	Review nearby childcare, schools, groceries, healthcare, recreation, and transport.	FR-NBH-02 to FR-NBH-10	None
UC-5.1	View Map and Nearby Places	Visitor; Configured Public Data Sources; OneMap	Explore categorised facilities and distances on a map.	FR-NBH-02 to FR-NBH-09; FR-TRV-01	None
UC-5.2	Check Commute Route	Visitor; OneMap	Request a route and travel estimate to a priority destination.	FR-TRV-02 to FR-TRV-05	None
UC-6.1	Calculate Family Suitability Score	No initiating actor	Compute an explainable neighbourhood score.	FR-DEC-01 to FR-DEC-03	Included by UC-4.1 and UC-6.2
UC-6.2	Compare Neighbourhoods	Visitor	Compare two to four neighbourhoods consistently.	FR-DEC-04; FR-DEC-01 to FR-DEC-03	Includes UC-6.1
UC-6.3	Manage Shortlist and Notes	Registered User	Save up to ten neighbourhoods and maintain private notes.	FR-DEC-05 to FR-DEC-07	None
UC-7.1	Monitor Automatic Data Synchronisation	Data Administrator; Configured Public Data Sources	Review status and history of automatic API updates.	FR-ADM-01 to FR-ADM-04	Monitoring only; synchronisation is automatic

7.4 Functional Requirement Traceability

FR ID	Use Case or Binding	Coverage
FR-ACC-01	UC-1.1	Directly covered
FR-ACC-02	UC-1.2	Directly covered
FR-ACC-03	UC-1.2	Directly covered
FR-ACC-04	UC-1.3	Directly covered
FR-ACC-05	UC-1.4	Directly covered
FR-ACC-06	UC-1.5	Directly covered
FR-PREF-01	UC-2.1	Directly covered
FR-PREF-02	UC-2.1	Directly covered

FR ID	Use Case or Binding	Coverage
FR-PREF-03	UC-2.1	Directly covered
FR-PREF-04	UC-2.1	Directly covered
FR-PREF-05	UC-2.1	Directly covered
FR-PREF-06	UC-2.1	Directly covered
FR-PREF-07	UC-2.1	Directly covered
FR-DISC-01	UC-3.1	Directly covered
FR-DISC-02	UC-3.1	Directly covered
FR-DISC-03	UC-3.2	Directly covered
FR-DISC-04	UC-3.1	Directly covered
FR-DISC-05	UC-3.1	Directly covered
FR-DISC-06	UC-3.1	Directly covered
FR-DISC-07	UC-3.1	Directly covered
FR-NBH-01	UC-4.1	Directly covered
FR-NBH-02	UC-4.2, UC-5.1	Directly covered
FR-NBH-03	UC-4.2, UC-5.1	Directly covered
FR-NBH-04	UC-4.2, UC-5.1	Directly covered
FR-NBH-05	UC-4.2, UC-5.1	Directly covered
FR-NBH-06	UC-4.2, UC-5.1	Directly covered
FR-NBH-07	UC-4.2, UC-5.1	Directly covered
FR-NBH-08	UC-4.2, UC-5.1	Directly covered
FR-NBH-09	UC-4.2, UC-5.1	Directly covered
FR-NBH-10	UC-4.2, UC-5.1	Directly covered
FR-TRV-01	UC-5.1	Directly covered
FR-TRV-02	UC-5.2	Directly covered
FR-TRV-03	UC-5.2	Directly covered
FR-TRV-04	UC-5.2	Directly covered
FR-TRV-05	UC-5.2	Directly covered
FR-DEC-01	UC-6.1	Directly covered
FR-DEC-02	UC-6.1	Directly covered
FR-DEC-03	UC-6.1	Directly covered
FR-DEC-04	UC-6.2	Directly covered
FR-DEC-05	UC-6.3	Directly covered
FR-DEC-06	UC-6.3	Directly covered
FR-DEC-07	UC-6.3	Directly covered
FR-ADM-01	UC-7.1	Directly covered
FR-ADM-02	UC-7.1	Directly covered
FR-ADM-03	Supporting system behaviour	Supporting or cross-cutting behaviour

FR ID	Use Case or Binding	Coverage
FR-ADM-04	UC-7.1	Directly covered
FR-RES-01	Supporting system behaviour	Supporting or cross-cutting behaviour
FR-RES-02	Cross-cutting	Supporting or cross-cutting behaviour
FR-RES-03	Cross-cutting	Supporting or cross-cutting behaviour

7.5 Use Case Descriptions

Each use case below follows the course template. Main flows have at most seven steps; alternative flows are numbered AF-<step><letter> and branch from that main-flow step. "System" means the FamPlan application. Inherited associations are not repeated: a Registered User can also perform every Visitor use case, and a Data Administrator every Registered User use case. "Notes and Issues" names the screen and endpoint that implement each use case, for traceability to the code.

7.5.1 UC-1.1 Register Account

Use Case ID	UC-1.1
Use Case Name	Register Account
Actor(s)	Visitor
Description	Lets a visitor create a FamPlan account using a display name, email address and password, after explicitly consenting to the stated data use.
Preconditions	1. The visitor is not signed in.
Postconditions	• Success: the account and an empty family preference profile are created, and the visitor is signed in as a Registered User. • Failure: no account is created; field-specific errors are shown.
Priority	Must
Frequency of Use	Low (once per user)
Flow of Events	1. Visitor opens the Sign in page and selects the Create account tab. 2. Visitor enters a display name, email address and password. 3. Visitor ticks the data-use consent statement and selects Create account. 4. System validates every field, the password policy and the consent. 5. System creates the account and stores only a salted Argon2id hash of the password. 6. System starts an authenticated session and shows the signed-in view.
Alternative Flows	• AF-4a Email already registered: System shows "An account with this email already exists. Sign in or reset your password."; return to step 2. • AF-4b Malformed email or password that breaks the policy: System shows a field error naming the rule; return to step 2. • AF-4c Consent not given: System shows a consent error; return to step 3.
Exceptions	• Too many attempts or service unavailable: System shows a generic error and creates no account.
Includes / Extends	None
Requirements Traced	FR-ACC-01
Special Requirements	NFR-SEC-01 (HTTPS only); NFR-SEC-02 (salted password hash); NFR-SEC-05 (input validation); NFR-PRIV-02 (no exact dates of birth or unnecessary personal data).
Assumptions	The email address uniquely identifies an account; there is no separate email-verification step in this release.
Notes and Issues	Implemented in client/src/pages/SignIn.tsx and POST /auth/register (server/src/account/routes.ts).

7.5.2 UC-1.2 Login and Logout

Use Case ID	UC-1.2
Use Case Name	Login and Logout
Actor(s)	Registered User
Description	Lets a registered user start an authenticated session with email and password, and end that session.
Preconditions	1. Login: the user has a registered account and is not signed in. 2. Logout: the user is signed in.
Postconditions	• Login success: a session is created (12-hour lifetime) and stored in an httpOnly cookie. • Login failure: a generic failure is shown and no session is created. • Logout: the session is invalidated on the server and the user is returned to a public page.
Priority	Must
Frequency of Use	High (every visit)
Flow of Events	1. User opens the Sign in page. 2. User enters email and password and selects Sign in. 3. System verifies the password against the stored hash and checks that the account is not locked. 4. System creates a session, resets the failed-login counter and shows the signed-in view. 5. Later, the user selects Log out. 6. System invalidates the session on the server and returns the user to a public page.
Alternative Flows	• AF-3a Wrong password or unknown email: System shows the same generic failure for both, so account existence is not revealed, and increments the failed-login counter; return to step 2. • AF-3b Fifth consecutive failure: System locks password sign-in for 15 minutes and shows the generic failure. • AF-5a Session expired: protected pages ask the user to sign in again.
Exceptions	• Service unavailable: System shows a generic error and creates no session.
Includes / Extends	None
Requirements Traced	FR-ACC-02, FR-ACC-03
Special Requirements	NFR-SEC-01 (HTTPS only); NFR-SEC-02 (password hashing); NFR-SEC-04 (15-minute lockout after five failures).
Assumptions	Users sign in from a browser that accepts first-party cookies.
Notes and Issues	Implemented in client/src/pages/SignIn.tsx, POST /auth/login and /auth/logout, and server/src/account/sessions.ts.

7.5.3 UC-1.3 Reset Password

Use Case ID	UC-1.3
Use Case Name	Reset Password
Actor(s)	Registered User (primary); Email Delivery Service (supporting)
Description	Lets a user who has forgotten the password receive a time-limited, single-use reset link by email and choose a new password.
Preconditions	1. The user is not signed in.
Postconditions	• Success: the new password hash is stored, the link is marked used, all existing sessions are ended and any lockout is cleared. • Failure: the password is unchanged.
Priority	Should
Frequency of Use	Low
Flow of Events	1. User selects Forgot password and enters the account email address. 2. System shows the same generic confirmation whether or not the email is registered. 3. If the email is registered, System asks the Email Delivery Service to send a reset link valid for 30 minutes. 4. User opens the link, enters a new password and confirms it. 5. System checks that the link is valid, unused and unexpired, and validates the password policy. 6. System stores the new password hash, marks the link used, ends existing sessions and asks the user to sign in.
Alternative Flows	• AF-5a Link invalid, already used or expired: System shows "This reset link is invalid, already used, or expired. Request a new link."; return to step 1. • AF-5b Password breaks the policy or the confirmation does not match: System shows a field error; return to step 4.
Exceptions	• Email delivery fails: no link arrives; the user can request a new link from step 1.
Includes / Extends	None
Requirements Traced	FR-ACC-04
Special Requirements	NFR-SEC-02 (password hashing); NFR-SEC-04 (a successful reset clears the lockout); NFR-SEC-05 (input validation).
Assumptions	The user still has access to the registered email inbox.
Notes and Issues	Implemented in client/src/pages/PasswordReset.tsx, POST /auth/forgot-password and /auth/reset-password, and server/src/account/mailer.ts.

7.5.4 UC-1.4 Manage Account Details

Use Case ID	UC-1.4
Use Case Name	Manage Account Details
Actor(s)	Registered User
Description	Lets a signed-in user change the display name stored on the account. Email address and password cannot be changed through this function.
Preconditions	1. The user is signed in.
Postconditions	• Success: the new display name is stored and shown. • Failure: the display name is unchanged and a field error is shown.
Priority	Must
Frequency of Use	Low
Flow of Events	1. User opens Account settings. 2. System shows the current display name and the read-only email address. 3. User edits the display name and saves. 4. System validates the new name. 5. System stores it and confirms the change.
Alternative Flows	• AF-4a Name empty or longer than 60 characters: System shows a field error; return to step 3.
Exceptions	• Session expired: System asks the user to sign in again.
Includes / Extends	None
Requirements Traced	FR-ACC-05
Special Requirements	NFR-SEC-05 (input validation).
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Account.tsx and PATCH /account.

7.5.5 UC-1.5 Delete Account

Use Case ID	UC-1.5
Use Case Name	Delete Account
Actor(s)	Registered User
Description	Permanently deletes the account and all associated personal data after explicit confirmation.
Preconditions	1. The user is signed in.
Postconditions	• Success: the account, credentials, sessions, saved preferences, priority destinations, and owned shortlist and notes are deleted; the session ends and an acknowledgement is shown. • Cancelled: nothing changes.
Priority	Must
Frequency of Use	Very low
Flow of Events	1. User opens Account settings and selects Delete account. 2. System explains exactly what will be deleted and that deletion cannot be undone. 3. User types DELETE to confirm and submits. 4. System deletes the account and all associated personal data. 5. System ends the session and shows a deletion acknowledgement on a public page.
Alternative Flows	• AF-3a User cancels: nothing is deleted. • AF-4a Confirmation text is not DELETE: System shows "Type DELETE to confirm account deletion."; return to step 3.
Exceptions	• Service unavailable: no data is deleted; System shows a generic error.
Includes / Extends	None
Requirements Traced	FR-ACC-06
Special Requirements	NFR-PRIV-01 (deleted personal data is no longer retrievable).
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Account.tsx and DELETE /account.

7.5.6 UC-2.1 Manage Family Preferences

Use Case ID	UC-2.1
Use Case Name	Manage Family Preferences
Actor(s)	Registered User
Description	Lets a user save family-planning stages, preferred planning areas, up to three priority destinations each with a travel mode, amenity distance thresholds and eight criterion weights, with optional stage-based suggested weights.
Preconditions	1. The user is signed in (needed to save).
Postconditions	• Success: the preference profile is saved and used by search, scoring and comparison; every saved destination contributes to the commute score using its own travel mode. • Failure: the previously saved profile is kept and field errors are shown.
Priority	Must
Frequency of Use	Medium (at setup, then occasionally)
Flow of Events	1. User opens Family preferences. 2. System loads the saved profile, or defaults for a new account. 3. User selects family stages and preferred planning areas. 4. User adds up to three priority destinations (label and address), chooses a travel mode for each (public transport, walk, car or cycle), and sets amenity distances and the eight criterion weights (0 to 5). 5. Optionally, user requests suggested weights; System fills stage-based presets without overwriting weights the user has already confirmed. 6. User saves; System validates every value. 7. System stores the profile and confirms the save.
Alternative Flows	• AF-1a Visitor not signed in: weights can be tried for the current session only, and System prompts the user to sign in to save. • AF-4a A fourth destination is added: System rejects it (maximum three). • AF-4b No travel mode chosen for a destination: System uses public transport. • AF-6a Weight outside 0 to 5 or not a whole number, an unsupported travel mode, or an unresolved destination address: System shows a field error; return to step 4.
Exceptions	• Address search unavailable: destinations cannot be added, but the other preferences can still be saved.
Includes / Extends	Saved values are reused by UC-3.2 Apply Saved Preferences.
Requirements Traced	FR-PREF-01 to FR-PREF-07
Special Requirements	NFR-PRIV-02 (family stages only; no exact dates of birth); NFR-SEC-05 (input validation).
Assumptions	A household is represented by a single preference profile.
Notes and Issues	Implemented in client/src/pages/Preferences.tsx, GET/PUT /preferences and POST /preferences/suggest-weights. Destination address suggestions come from OneMap search through GET /geocode.

7.5.7 UC-3.1 Search Neighbourhoods

Use Case ID	UC-3.1
Use Case Name	Search Neighbourhoods
Actor(s)	Visitor
Description	Finds neighbourhoods by town, planning area, estate, street address or postal code, and filters and sorts the results. An address or postal code is resolved to one exact location, and distances, counts and scores are measured from that coordinate.
Preconditions	1. Validated neighbourhood data has been loaded.
Postconditions	• Success: matching neighbourhood profiles are listed and plotted on the map, each labelled as a neighbourhood profile rather than a property listing. • Failure: an empty state or field-specific correction is shown.
Priority	Must
Frequency of Use	High
Flow of Events	1. Visitor opens Discover neighbourhoods. 2. Visitor enters a town, estate, street address or six-digit postal code, or picks planning areas. 3. Visitor optionally sets filters (family stage, amenity category, minimum facility count, facility radius, commute destination and time) and a sort order. 4. System validates the criteria. 5. For an address or postal code, System resolves the text to one exact location and uses its coordinate as the origin for the radius, counts, score evidence and routes. 6. System searches the locally stored validated data and orders the matches by the chosen sort. 7. System lists the matching neighbourhood profiles, states which origin was used, and plots them on the map.
Alternative Flows	• AF-2a Spelling variant: System still matches the intended area by approximate name matching. • AF-3a Registered User selects Apply my preferences: UC-3.2 extends the search at this point. • AF-4a Invalid or contradictory criteria, or a postal code with fewer than six digits: System shows field-specific correction guidance (for example "Enter all six digits of the Singapore postal code, including any leading zero."); return to step 3. • AF-5a Several places match the address: System lists the matching addresses and asks the visitor to choose one; continue at step 6 with the chosen location. • AF-5b No Singapore address matches: System shows "No Singapore address matches this postal code or address. Check the input and try again."; return to step 2. • AF-7a No matches: System shows an empty state suggesting wider filters.
Exceptions	• Address search unavailable: System falls back to town and planning-area name matching where the text matches a name; otherwise it shows "Address search is temporarily unavailable. Search by town or area name instead." • Source data older than the freshness threshold: results are still shown with a stale-data warning.
Includes / Extends	Extended by UC-3.2 (extension point: step 3, Apply my preferences).
Requirements Traced	FR-DISC-01, FR-DISC-02, FR-DISC-04 to FR-DISC-07
Special Requirements	NFR-PERF-01 (search response time); NFR-REL-03 (search reads local validated data, never live APIs); NFR-SEC-05 (input validation).
Assumptions	Neighbourhoods are planning-area subzones from the URA dataset.
Notes and Issues	Implemented in client/src/pages/Explore.tsx and GET /neighbourhoods (searchNeighbourhoods() in server/src/neighbourhoods/service.ts). Address resolution uses OneMap search through the geocode cache.

7.5.8 UC-3.2 Apply Saved Preferences

Use Case ID	UC-3.2
Use Case Name	Apply Saved Preferences
Actor(s)	Registered User
Description	Fills the current search criteria from the user's saved family preferences and refreshes the results.
Preconditions	1. The user is signed in. 2. A search is in progress (UC-3.1).
Postconditions	• Search criteria are populated and results refreshed; the saved preference values are not changed.
Priority	Must
Frequency of Use	Medium
Flow of Events	1. During a search, the user selects Apply my preferences. 2. System loads the saved family stages, preferred areas, amenity distances, priority destinations with their travel modes, and weights. 3. System fills the search criteria with these values. 4. System re-runs the search (UC-3.1 steps 4 to 7). 5. System confirms that the preferences were applied without changing the saved values.
Alternative Flows	• AF-1a User not signed in: the control reads "Sign in to apply saved family preferences". • AF-2a Profile only partly filled: System applies the saved fields and keeps the current values for the rest. • AF-2b No saved profile: System directs the user to Family preferences (UC-2.1).
Exceptions	• None.
Includes / Extends	Extends UC-3.1.
Requirements Traced	FR-DISC-03
Special Requirements	None.
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Explore.tsx (applyMyPreferences()).

7.5.9 UC-4.1 View Neighbourhood Profile

Use Case ID	UC-4.1
Use Case Name	View Neighbourhood Profile
Actor(s)	Visitor
Description	Shows a selected neighbourhood's summary (name, planning area, map location and coverage basis) together with its explainable family suitability score.
Preconditions	1. A neighbourhood has been selected from results, a comparison, the shortlist or a direct link.
Postconditions	• The profile and score breakdown are displayed.
Priority	Must
Frequency of Use	High
Flow of Events	1. Visitor selects a neighbourhood. 2. System loads it from the local validated data. 3. System displays its name, planning area, map location and boundary or coverage basis. 4. System calculates its family suitability score (includes UC-6.1). 5. System shows the score with its criterion breakdown and the statement that the score is a comparison aid, not a guarantee. 6. System shows the data sources and their update times.
Alternative Flows	• AF-2a Unknown neighbourhood: System shows "not found" with a link back to search. • AF-4a A criterion lacks data: the score discloses how its weight was treated (see UC-6.1).
Exceptions	• Cached data older than the freshness threshold: System shows its age and a stale-data warning.
Includes / Extends	Includes UC-6.1.
Requirements Traced	FR-NBH-01; FR-DEC-01 to FR-DEC-03
Special Requirements	NFR-TRANS-01 and NFR-TRANS-02 (explainable score, limitation statement); NFR-DATA-01 (provenance on every result).
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Profile.tsx and GET /neighbourhoods/:id.

7.5.10 UC-4.2 Explore Family Amenities

Use Case ID	UC-4.2
Use Case Name	Explore Family Amenities
Actor(s)	Visitor (primary); Configured Public Data Sources (supporting)
Description	Lets the visitor review nearby childcare centres, kindergartens, schools, supermarkets, clinics, parks, MRT stations and bus stops within a chosen radius, with counts, nearest facilities, distances and sources.
Preconditions	1. A neighbourhood profile is open (UC-4.1).
Postconditions	• Per-category facilities, counts and availability states are displayed.
Priority	Must
Frequency of Use	High
Flow of Events	1. Visitor chooses a radius (500 m, 1 km or 2 km) and optionally an amenity category. 2. System selects facilities within the radius from the validated datasets synchronised from the configured public data sources. 3. System shows the count for each category and the nearest facility with its distance and calculation basis. 4. Visitor opens a facility. 5. System shows its name, category, address where available, source and update time. 6. For schools and childcare, System states that proximity does not guarantee admission, enrolment, vacancy or eligibility.
Alternative Flows	• AF-2a The same facility appears in two sources: System counts it once. • AF-3a No facility within the radius: System shows an explicit "no match" state instead of a misleading zero. • AF-3b A category's source is unavailable, incomplete or stale: System shows that status for the category; other categories remain visible.
Exceptions	• None beyond AF-3b; data is served from the local validated store.
Includes / Extends	None
Requirements Traced	FR-NBH-02 to FR-NBH-10
Special Requirements	NFR-DATA-01 (provenance); NFR-DATA-02 (31-day freshness threshold); NFR-DATA-04 (de-duplicated facility records).
Assumptions	Distances are measured from the neighbourhood reference point (straight line).
Notes and Issues	Implemented in client/src/pages/Profile.tsx, GET /neighbourhoods/:id/facilities and server/src/neighbourhoods/dedupe.ts.

7.5.11 UC-5.1 View Map and Nearby Places

Use Case ID	UC-5.1
Use Case Name	View Map and Nearby Places
Actor(s)	Visitor (primary); Configured Public Data Sources and OneMap (supporting)
Description	Shows the neighbourhood and its matched facilities on an interactive map with distinguishable category markers.
Preconditions	1. A neighbourhood profile is open (UC-4.1).
Postconditions	• The map shows the neighbourhood boundary, reference point and categorised facility markers.
Priority	Must
Frequency of Use	High
Flow of Events	1. Visitor opens the map on the neighbourhood profile. 2. System draws the OneMap base map with the neighbourhood boundary and reference point. 3. System plots the matched facilities within the radius with a distinct marker for each category. 4. Visitor toggles map layers or changes the radius. 5. System updates the markers and counts. 6. Visitor selects a marker; System shows the facility details and distance.
Alternative Flows	• AF-3a Facility without coordinates: it is listed but not plotted, with a note. • AF-3b Overlapping markers: each remains individually selectable.
Exceptions	• Base map unavailable: the facility list and counts remain usable.
Includes / Extends	None
Requirements Traced	FR-NBH-02 to FR-NBH-09; FR-TRV-01
Special Requirements	NFR-ACC-01 (keyboard-accessible map controls); NFR-REL-03 (one dependency failing does not block others).
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Profile.tsx (map layers) and client/src/components/map.tsx.

7.5.12 UC-5.2 Check Commute Route

Use Case ID	UC-5.2
Use Case Name	Check Commute Route
Actor(s)	Visitor (primary); OneMap (supporting)
Description	Requests a route and travel estimate from the neighbourhood to a priority destination for a supported travel mode, optionally asking for a barrier-free route.
Preconditions	1. A neighbourhood profile is open. 2. A destination is chosen (saved priority destination or entered address).
Postconditions	• A route with distance, duration, provider and retrieval time is shown, or an explained unavailable state.
Priority	Must
Frequency of Use	Medium
Flow of Events	1. Visitor opens Commute & routes from a neighbourhood profile. 2. Visitor chooses a saved priority destination or searches another place; a saved destination preselects its saved travel mode. 3. Visitor chooses a travel mode (public transport, walk, car or cycle) and optionally ticks Barrier-free routes only. 4. System checks that the mode is supported by the routing provider. 5. System requests the route from OneMap. 6. System shows the route on the map with distance, duration, travel mode, provider and retrieval time.
Alternative Flows	• AF-3a Mode differs from the destination's saved mode: System labels the route as a preview only and states that the commute score still uses the saved mode. • AF-4a Unsupported mode: System explains the rejection; return to step 3. • AF-5a Barrier-free route requested: System states that step-free coverage is unavailable from the provider and advises checking lift and ramp availability with LTA or the station operator. • AF-5b OneMap finds no route: System shows a "no route" state with a labelled straight-line estimate.
Exceptions	• OneMap unavailable, timed out or not configured: System shows a labelled straight-line estimate instead; the rest of the profile still works.
Includes / Extends	None
Requirements Traced	FR-TRV-02 to FR-TRV-05
Special Requirements	NFR-REL-03 (routing failure is isolated); NFR-DATA-01 (provider and retrieval time shown).
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Commute.tsx, GET /neighbourhoods/:id/route and server/src/travel/route.ts and onemap.ts.

7.5.13 UC-6.1 Calculate Family Suitability Score

Use Case ID	UC-6.1
Use Case Name	Calculate Family Suitability Score
Actor(s)	None (included behaviour)
Description	Computes an explainable 0 to 100 family suitability score from eight criteria using the user's weights and the versioned scoring configuration.
Preconditions	1. Invoked by UC-4.1 or UC-6.2 with a neighbourhood and a set of weights (saved, session or default).
Postconditions	• A total score is returned with each criterion's value, normalised value, weight and contribution, any missing-data notes, and the configuration and dataset versions used.
Priority	Must
Frequency of Use	Very high (every profile and comparison)
Flow of Events	1. The base use case supplies the neighbourhood and the weights. 2. System gathers evidence for the eight criteria from the local validated data; the commute value is the average estimated minutes to every priority destination, each by its saved travel mode. 3. System normalises each value with its criterion's rule (facility count, nearest distance or commute minutes). 4. System excludes criteria that lack data and re-normalises the remaining weights. 5. System computes each contribution and the total, rounding an exact 0.5 upward. 6. System returns the total with the itemised breakdown and version stamps.
Alternative Flows	• AF-2a No priority destination is set: the commute criterion lacks data and is handled as in step 4. • AF-4a All weights are zero: System weights every available criterion equally and discloses this. • AF-4b No criterion has data: the score is shown as unavailable with an explanation.
Exceptions	• None.
Includes / Extends	Included by UC-4.1 and UC-6.2.
Requirements Traced	FR-DEC-01 to FR-DEC-03
Special Requirements	NFR-DATA-03 (identical inputs and versions reproduce the same score); NFR-TRANS-01 (itemised explanation); NFR-MAINT-01 (rules kept in versioned configuration).
Assumptions	None.
Notes and Issues	Implemented in server/src/scoring/score.ts (calculateScore()) and server/src/scoring/config.ts (SCORING_CONFIG).

7.5.14 UC-6.2 Compare Neighbourhoods

Use Case ID	UC-6.2
Use Case Name	Compare Neighbourhoods
Actor(s)	Visitor
Description	Compares two to four selected neighbourhoods side by side using the same weights and radius.
Preconditions	1. At least two neighbourhoods have been selected for comparison.
Postconditions	• A side-by-side comparison of scores and family information is displayed.
Priority	Must
Frequency of Use	Medium
Flow of Events	1. Visitor marks two to four neighbourhoods for comparison from results, profiles or the shortlist. 2. Visitor opens Compare neighbourhoods. 3. System validates the selection. 4. System calculates each neighbourhood's score with the same weights and radius (includes UC-6.1). 5. System shows them side by side: suitability match, childcare and kindergartens, schools, daily needs, public transport, accessibility and the commute estimate to each priority destination. 6. Visitor changes the radius or weights; System recalculates every column consistently.
Alternative Flows	• AF-3a Fewer than two or more than four selected: System shows "Choose two to four neighbourhoods to compare."; return to step 1. • AF-3b The same neighbourhood selected twice: System counts it once. • AF-5a A criterion lacks data for one neighbourhood: System marks it in that column.
Exceptions	• None.
Includes / Extends	Includes UC-6.1.
Requirements Traced	FR-DEC-04; FR-DEC-01 to FR-DEC-03
Special Requirements	NFR-PERF-02 (comparison response time); NFR-TRANS-01 and NFR-TRANS-02.
Assumptions	None.
Notes and Issues	Implemented in client/src/pages/Compare.tsx and GET /compare.

7.5.15 UC-6.3 Manage Shortlist and Notes

Use Case ID	UC-6.3
Use Case Name	Manage Shortlist and Notes
Actor(s)	Registered User
Description	Lets a user save up to ten neighbourhoods to a shortlist, remove them, and keep a private note of up to 500 characters on each.
Preconditions	1. The user is signed in.
Postconditions	• The shortlist and notes are saved and appear again after the next sign-in.
Priority	Must
Frequency of Use	Medium
Flow of Events	1. User selects Add to shortlist on a neighbourhood. 2. System checks for a duplicate and the ten-entry limit, then saves the entry. 3. User opens Your shortlist. 4. System lists the saved neighbourhoods with their current scores. 5. User writes, edits or clears a private note. 6. System validates and saves the note. 7. User removes an entry; System deletes it and confirms.
Alternative Flows	• AF-1a Visitor not signed in: System prompts the visitor to sign in or create an account. • AF-2a Neighbourhood already saved: System shows it as already saved and creates no duplicate. • AF-2b An eleventh entry: System rejects it and explains the capacity limit. • AF-6a Note longer than 500 characters: System shows a validation error and keeps the previous note.
Exceptions	• Session expired: System asks the user to sign in again; nothing is lost.
Includes / Extends	None
Requirements Traced	FR-DEC-05 to FR-DEC-07
Special Requirements	NFR-SEC-05 (notes are validated and rendered as plain text); NFR-PRIV-01 (notes are deleted with the account).
Assumptions	Notes are private to the owner.
Notes and Issues	Implemented in client/src/pages/Shortlist.tsx and the /shortlist endpoints. Entries are stored per account (ShortlistEntry, unique per account and neighbourhood).

7.5.16 UC-7.1 Monitor Automatic Data Synchronisation

Use Case ID	UC-7.1
Use Case Name	Monitor Automatic Data Synchronisation
Actor(s)	Data Administrator (primary); Configured Public Data Sources (supporting)
Description	Lets an administrator review the status and history of the automatic public-data synchronisation. Synchronisation itself runs automatically (FR-ADM-03); there is no manual sync action.
Preconditions	1. The user is signed in with the Data Administrator role.
Postconditions	• Source status and synchronisation history are displayed; no data is changed (read-only).
Priority	Must
Frequency of Use	Low (routine check)
Flow of Events	1. Administrator opens the Data sync dashboard. 2. System verifies the administrator role. 3. System shows each source's status, latest successful and latest attempted synchronisation, record count and freshness. 4. Administrator filters the synchronisation history by source or date. 5. System lists the runs newest first with start and end time, outcome, records added, changed and removed, and a sanitised error summary.
Alternative Flows	• AF-2a Signed-in user without the administrator role: System denies access (403). • AF-2b Not signed in: System asks the user to sign in. • AF-3a A source's latest run failed: System shows the failure while the previous validated dataset stays active.
Exceptions	• None.
Includes / Extends	None
Requirements Traced	FR-ADM-01, FR-ADM-02, FR-ADM-04 (background: FR-ADM-03, FR-RES-01)
Special Requirements	NFR-SEC-03 (role check at UI and API level); error summaries contain no secrets.
Assumptions	Synchronisation runs daily at 04:00 Singapore time (SYNC_CRON) and at start-up when a source is due.
Notes and Issues	Implemented in client/src/pages/Admin.tsx, GET /admin/sources and GET /admin/history (server/src/synchronisation/routes.ts).

8. Technology Stack Recommendation

This is a deliberately small recommendation rather than a binding requirement. It keeps the project understandable for the team and supports the client-server, mapping, scoring, persistence, testing, and automatic synchronisation work already required.

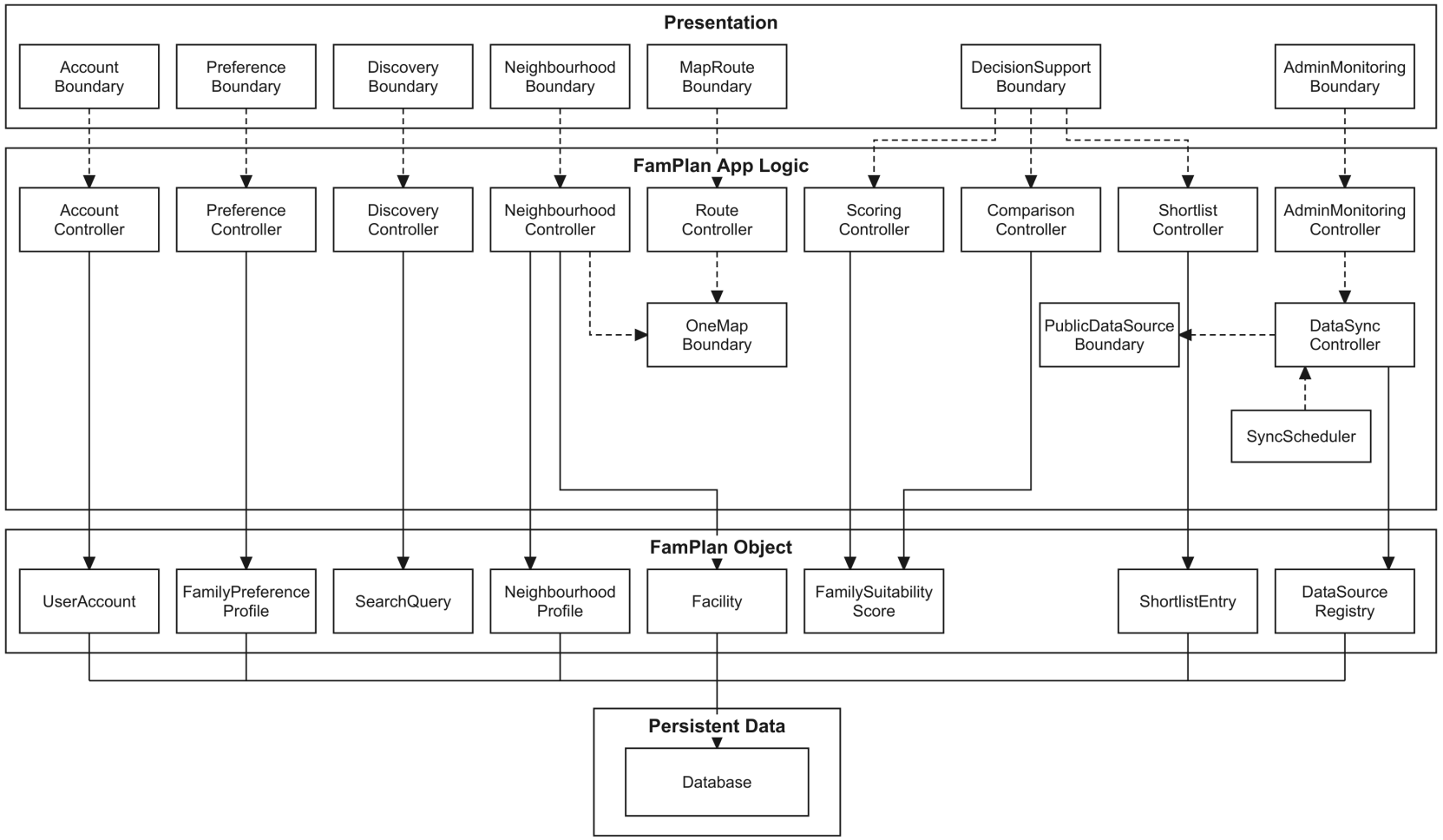
Layer	Recommendation	Reason
Frontend	React with TypeScript and Vite	One responsive browser interface with reusable components and strong typing.

Layer	Recommendation	Reason
Backend	Node.js with Express and TypeScript	Simple REST API and one language across client and server.
Database	PostgreSQL with Prisma	Relational data, geolocation fields, constraints, migrations, and straightforward querying.
Maps and routes	Leaflet with authorised OneMap services	Lightweight interactive maps without introducing a separate native application.
Automatic data sync	Vercel Cron in production; node-cron on a long-running server	Calls configured APIs at least daily, validates data, and records outcomes without an administrator trigger.
Testing	Vitest, Supertest, and Playwright	Covers calculation units, API behaviour, and a small number of end-to-end workflows.
Local setup and deployment	GitHub, Vercel and Neon PostgreSQL	Keeps development reproducible without microservices, Kubernetes, or extra infrastructure.

8.1 Architecture Boundary

Use one frontend, one backend, and one PostgreSQL database. Keep data-source adapters and scoring logic as modules inside the backend rather than separate services. The scheduled job calls the same validation and activation modules used by the application. Do not introduce microservices, message queues, machine learning, a native mobile application, or a separate administrator backend for the first release.

8.2 System Architecture



Layered architecture: dashed arrows are calls and dependencies; solid arrows show the entity each controller reads or updates; persistent entities are stored in PostgreSQL. Deployment: the React client is served by Vercel; Express runs as serverless Node.js functions in Singapore (sin1); Prisma connects to hosted PostgreSQL; one Vercel Cron job refreshes each of the 9 public data sources daily. Local development runs the same Express app as one long-running server with node-cron.

Presentation Layer
This layer is responsible for user interaction and view rendering inside the user's web browser, implemented as a TypeScript Single-Page Application using React and Vite. The boundary components capture user inputs, enforce form formatting, and delegate actions to backend REST API endpoints.
This layer consists of:

1. **AccountBoundary**: Presents registration, login, logout, display name updates, and password reset forms. Captures credentials and displays operation outcomes to users.
2. **PreferenceBoundary**: Captures family life stages, preferred planning areas, priority commute destinations with travel modes, and 0–5 criterion importance weights.
3. **DiscoveryBoundary**: Captures neighbourhood search queries and filter options (amenity distances, counts, and commute limits). Presents filterable and sortable result cards.
4. **NeighbourhoodBoundary**: Displays neighbourhood summary profiles, amenity counts by category, and availability states.
5. **MapRouteBoundary**: Renders an interactive Leaflet map marking categorized facilities, proximity radii (500 m, 1 km, 2 km), and travel route estimates.
6. **DecisionSupportBoundary**: Presents suitability score breakdowns, captures comparison selections (2 to 4 neighbourhoods), and displays the user's shortlisted neighbourhoods and private notes.
7. **AdminMonitoringBoundary**: Displays the automated synchronisation dashboard showing data source health, record counts, execution timestamps, and sync logs.

App Logic Layer
This layer contains all controller classes and external adapter boundaries executing on the Node.js / Express backend, as serverless functions in production. It coordinates business rules, processes data transformations, and encapsulates third-party integrations.
This layer consists of:

1. **AccountController**: Enforces password hashing using Argon2id, verifies session tokens, enforces login lockout policies (15 minutes after 5 failures), and manages account deletion lifecycles.
2. **PreferenceController**: Evaluates family planning stages, supplies stage-based weight presets, and manages persistent user preference profiles.
3. **DiscoveryController**: Validates search query bounds and executes spatial and attribute filters over cached neighbourhood datasets.
4. **NeighbourhoodController**: Aggregates nearby facilities within requested radius buffers and compiles the amenity summary for neighbourhood profiles.
5. **RouteController**: Validates transit modes and calculates commute estimates between neighbourhood reference points and priority destinations.
6. **ScoringController**: Implements the deterministic scoring engine, normalizes 8 criterion values, applies missing-data policies, and computes the rounded 0–100 suitability score.
7. **ComparisonController**: Validates that 2 to 4 neighbourhoods are selected and compiles side-by-side metric evaluations.
8. **ShortlistController**: Enforces the 10-entry shortlist capacity limit per user and validates private text notes up to 500 characters.
9. **AdminMonitoringController**: Enforces administrator role authorization and retrieves data source health and synchronisation audit logs.

10. **DataSyncController**: Fetches datasets from external endpoints, validates dataset schemas against checksums, creates provenance stamps, and persists validated snapshots.
11. **SyncScheduler**: Runs scheduled data synchronisation at least once every 24 hours without human intervention. It uses node-cron on a long-running server and one Vercel Cron job per source in the serverless production deployment.
12. **OneMapBoundary**: External adapter interfacing with the Singapore OneMap REST API for address geocoding, reverse geocoding, and transit routing.
13. **PublicDataSourceBoundary**: External adapter encapsulating public data providers (ECDA childcare, MOE schools, URA subzones, LTA DataMall) to insulate domain logic against schema changes.

App Object Layer
This layer consists of the core domain entity classes depicted in the architecture diagram:

1. **UserAccount**: Represents a registered user identity and encapsulates account status, timestamps, and role privileges.
2. **FamilyPreferenceProfile**: Stores user-configured family planning stages, preferred planning areas, and amenity distance criteria.
3. **SearchQuery**: Encapsulates transient request criteria, location text, and active filter thresholds during neighbourhood discovery.
4. **NeighbourhoodProfile**: Represents a comparable planning zone with name, planning area, reference point coordinates, and boundary basis.
5. **Facility**: Represents an individual school, clinic, preschool, supermarket, park, or transport station with geocoordinates and source metadata.
6. **FamilySuitabilityScore**: Encapsulates the overall 0–100 score, calculation configuration version, and contributing criterion breakdowns.
7. **ShortlistEntry**: Represents a saved neighbourhood pinned to a user account, supporting an optional private note.
8. **DataSourceRegistry**: Maintains configuration, freshness thresholds, and operational health status for each public data source.

Persistent Data Layer
This layer contains the database that will store all of the entities. In FamPlan, this is implemented using a PostgreSQL relational database to reliably manage and persist application data.

9. Class Diagram

If the images are unclear, please refer to the raw files uploaded together with this document.

9.1 Entity Model

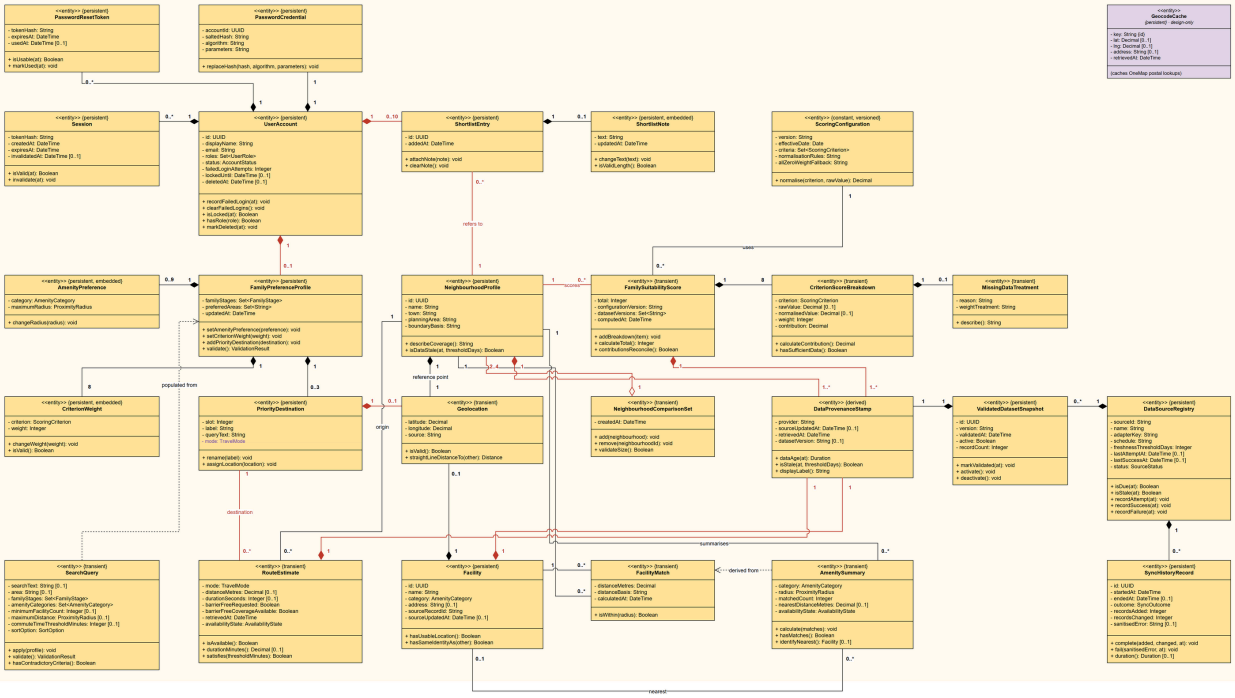
Domain data model defining 26 entity classes, attributes, relationships, and 11 enumerations implemented via Prisma/PostgreSQL. Each entity is tagged with its persistence decision ({persistent}, {transient}, ...); GeocodeCache is a design-only persistent class.

FamPlan Class Diagram v2.0 — Diagram A: Entity Model

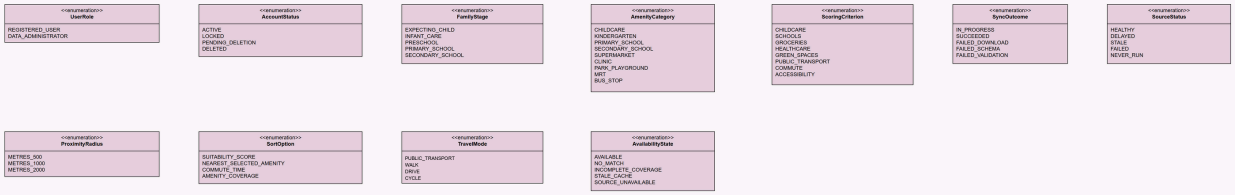
26 entity classes · 11 enumerations · 33 structural relationships · boundary and control classes are in Diagram B

Design refinement 16/10/2026: each entity is tagged with its persistence decision (Bruegge §7.4); GeocodeCache added as a design-only persistent entity · 31/10/2026: PriorityDestination.mode added; TravelMode values listed

ENTITY CLASSES



ENUMERATIONS



CONSTRAINT NOTES

Account & preferences
(UserAccount.email is unique)
(five consecutive failed logins lock password authentication for 15 minutes)
(CriterionWeight.weight is an integer from 0 to 5)
(one CriterionWeight per ScoringCriterion; eight weights in total)
(one AmenityPreference per AmenityCategory)
(maximum three PriorityDestination objects per profile)

Neighbourhood & amenities
(facility records representing the same named facility at the same location are not counted twice)
(school or childcare proximity does not guarantee admission, eligibility or vacancy)

Decision support & shortlist
(NeighbourhoodComparisonSet contains two to four distinct neighbourhoods)
(ShortlistEntry is unique by account and neighbourhood)
(maximum ten ShortlistEntry objects per UserAccount)
(ShortlistNote.text length is at most 500 characters)
(score is from 0 to 100; exact 0.5 rounds upward)

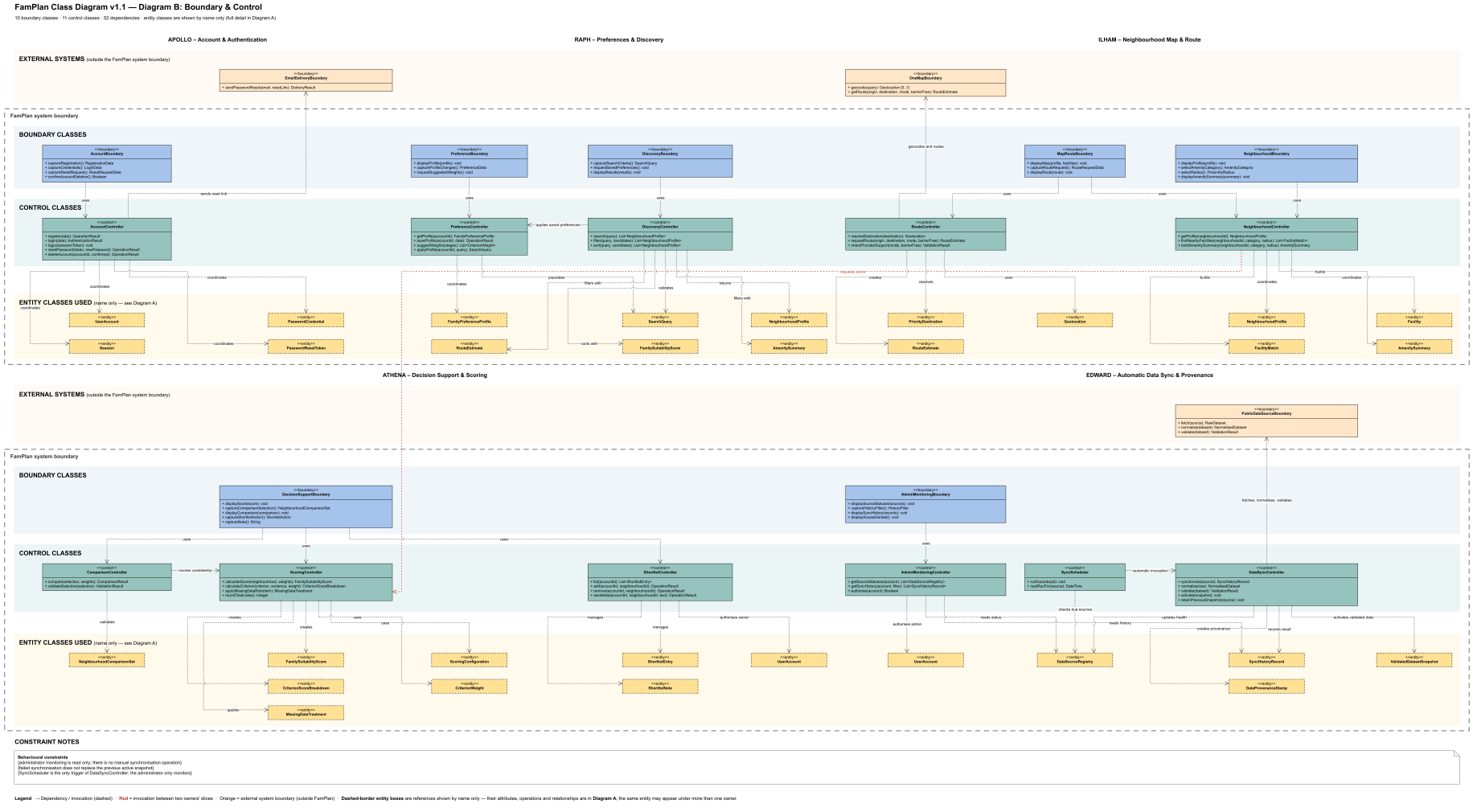
Synchronisation & provenance
(at most one active ValidatedDatasetSnapshot per DataSourceRegistry)
(failed synchronisation does not replace the previous active snapshot)

Legend ◆ Composition (lifecycle ownership) · ◇ Shared aggregation · —— Association (multiplicity at both ends) · → Dependency (dashed) · Red = link between two owners' slices · Enumerations are attribute types (no lines) · Boundary and control classes and their dependencies are in **Diagram B**. Actors are not classes: users are *UserAccount*; administrator authority is *UserRole.DATA_ADMINISTRATOR*.

Persistence **{persistent}** stored in PostgreSQL via Prisma (prisma/schema.prisma) · **{persistent, embedded}** stored as columns of its owner (CriterionWeight → FamilyPreferenceProfile.w*, AmenityPreference → amenityThresholds JSON, ShortlistNote → ShortlistEntry.note) · **{transient}** computed per request, never stored · **{derived}** built from stored rows · **{constant, versioned}** code constant SCORING_CONFIG · Prisma model names differ for NeighbourhoodProfile (*Neighbourhood*) and ValidatedDatasetSnapshot (*DatasetSnapshot*) · purple = added in the design model

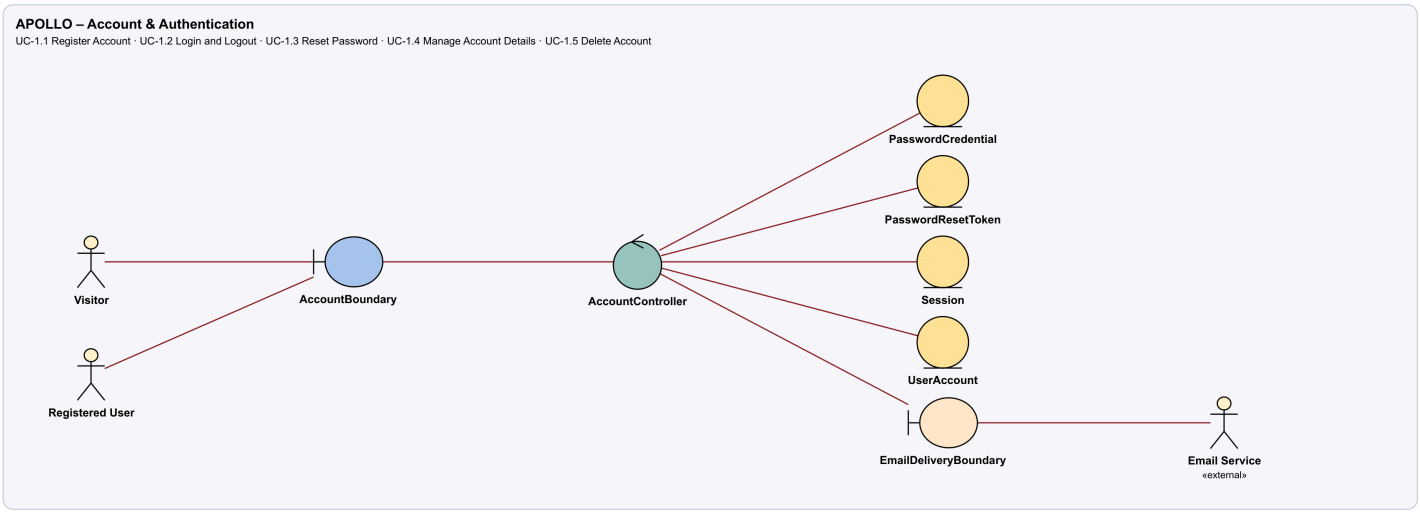
9.2 Boundary and Control Subsystems

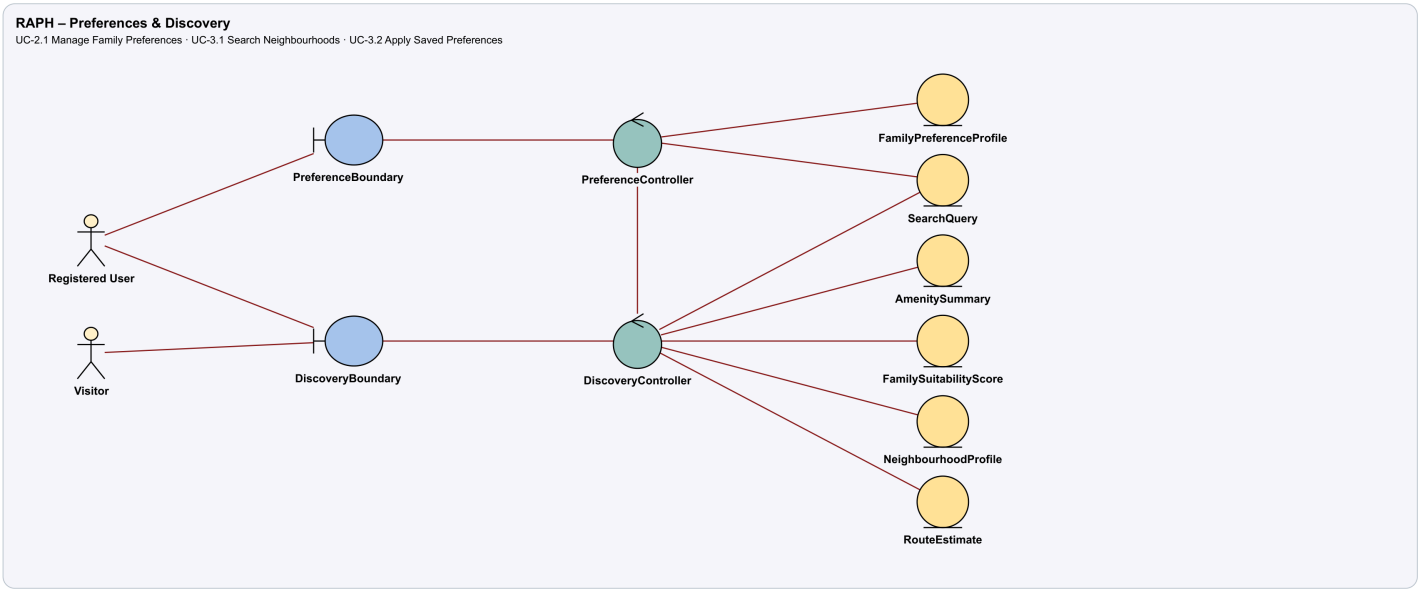
Structural view mapping UI boundaries to controllers across the 5 functional subsystems.

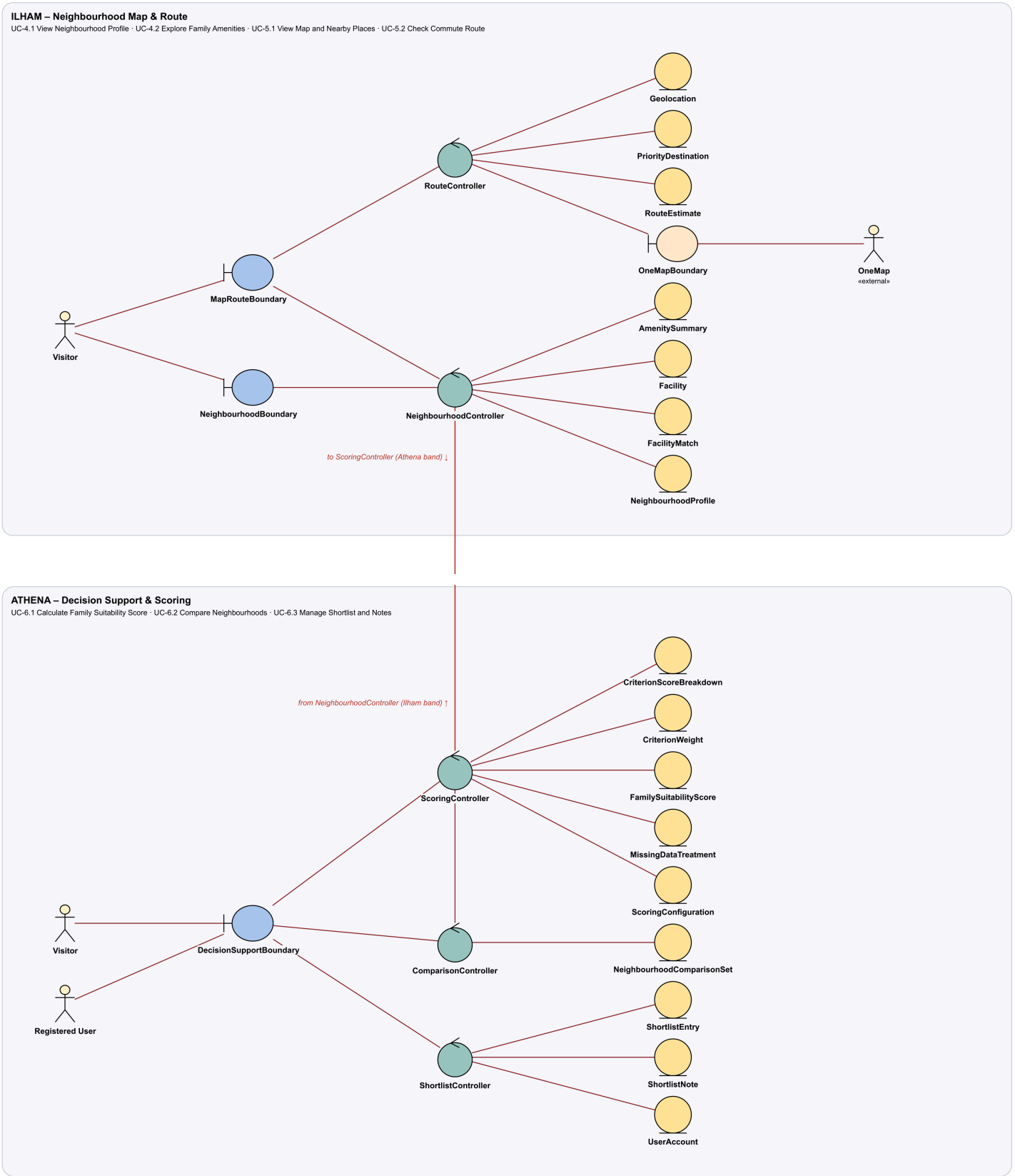


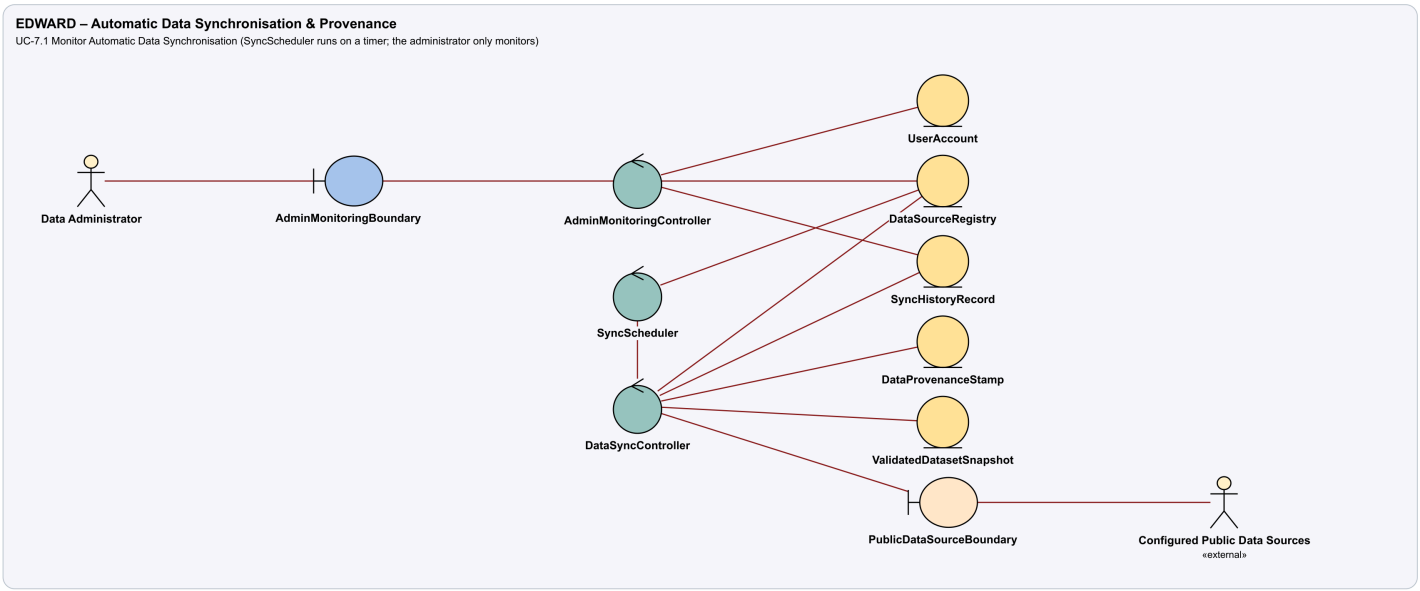
9.3 Robustness View

BCE analysis view illustrating communication between actors, external services, boundaries, controls and entity stores.






ILHAM – Neighbourhood Map & Route
UC-4.1 View Neighbourhood Profile · UC-4.2 Explore Family Amenities · UC-5.1 View Map and Nearby Places · UC-5.2 Check Commute Route
Geolocation
PriorityDestination
RouteController
RouteEstimate
OneMapBoundary
OneMap
«external»
MapRouteBoundary
Visitor
AmenitySummary
Facility
NeighbourhoodBoundary
NeighbourhoodController
FacilityMatch
to ScoringController (Athena band) ↓
NeighbourhoodProfile
ATHENA – Decision Support & Scoring
UC-6.1 Calculate Family Suitability Score · UC-6.2 Compare Neighbourhoods · UC-6.3 Manage Shortlist and Notes
from NeighbourhoodController (Ilham band) ↑
CriterionScoreBreakdown
CriterionWeight
ScoringController
FamilySuitabilityScore
MissingDataTreatment
ScoringConfiguration
Visitor
DecisionSupportBoundary
ComparisonController
NeighbourhoodComparisonSet
Registered User
ShortlistEntry
ShortlistController
ShortlistNote
UserAccount



Legend ♀ Actor · ⊢◯ Boundary (orange = external system interface, outside FamPlan) · ↻◯ Control · ◯ Entity · Lines are communication links: actors talk only to boundaries, boundaries to controls, controls to entities, other controls and external boundaries. **Red** = link between two owners' bands. An actor or entity may appear in more than one band. Registered User specialises Visitor and Data Administrator specialises Registered User (see the use case diagram).

9.4 Design Layer

Design-level classes added when refining the analysis model (Lab 3): a start-up class, container classes, persistence, access control and design-pattern roles. Grey dashed boxes are classes drawn in Sections 9.1 and 9.2; file paths are relative to prototype/server/src.

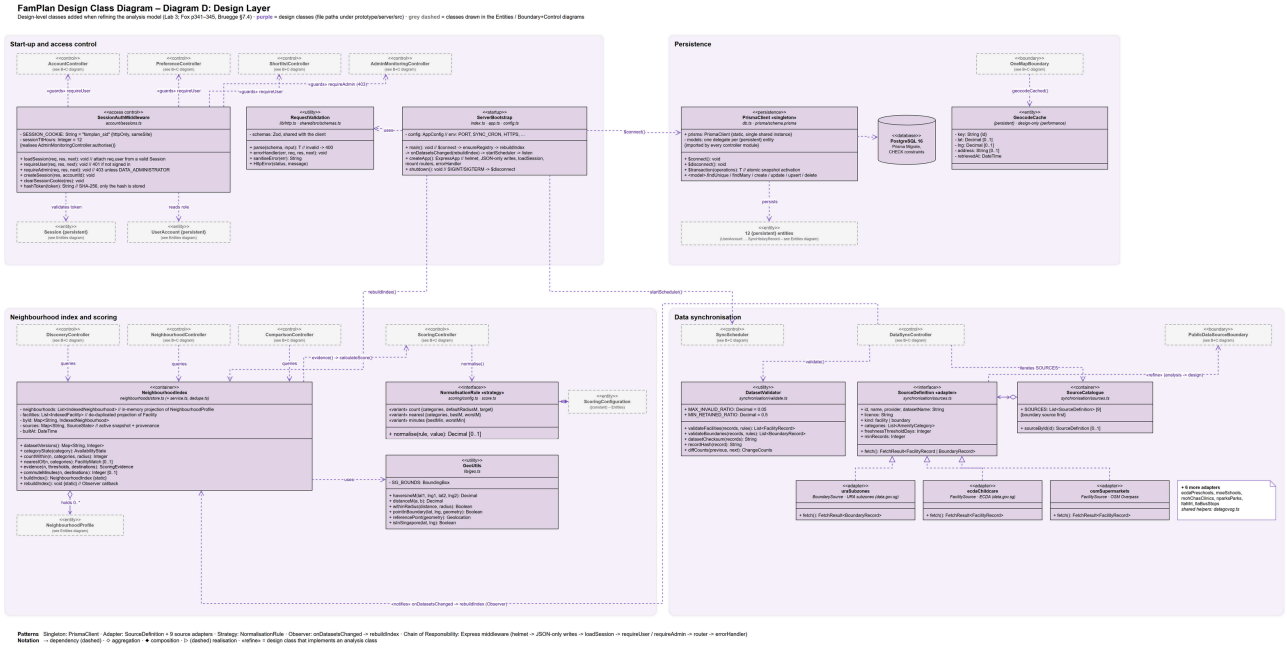
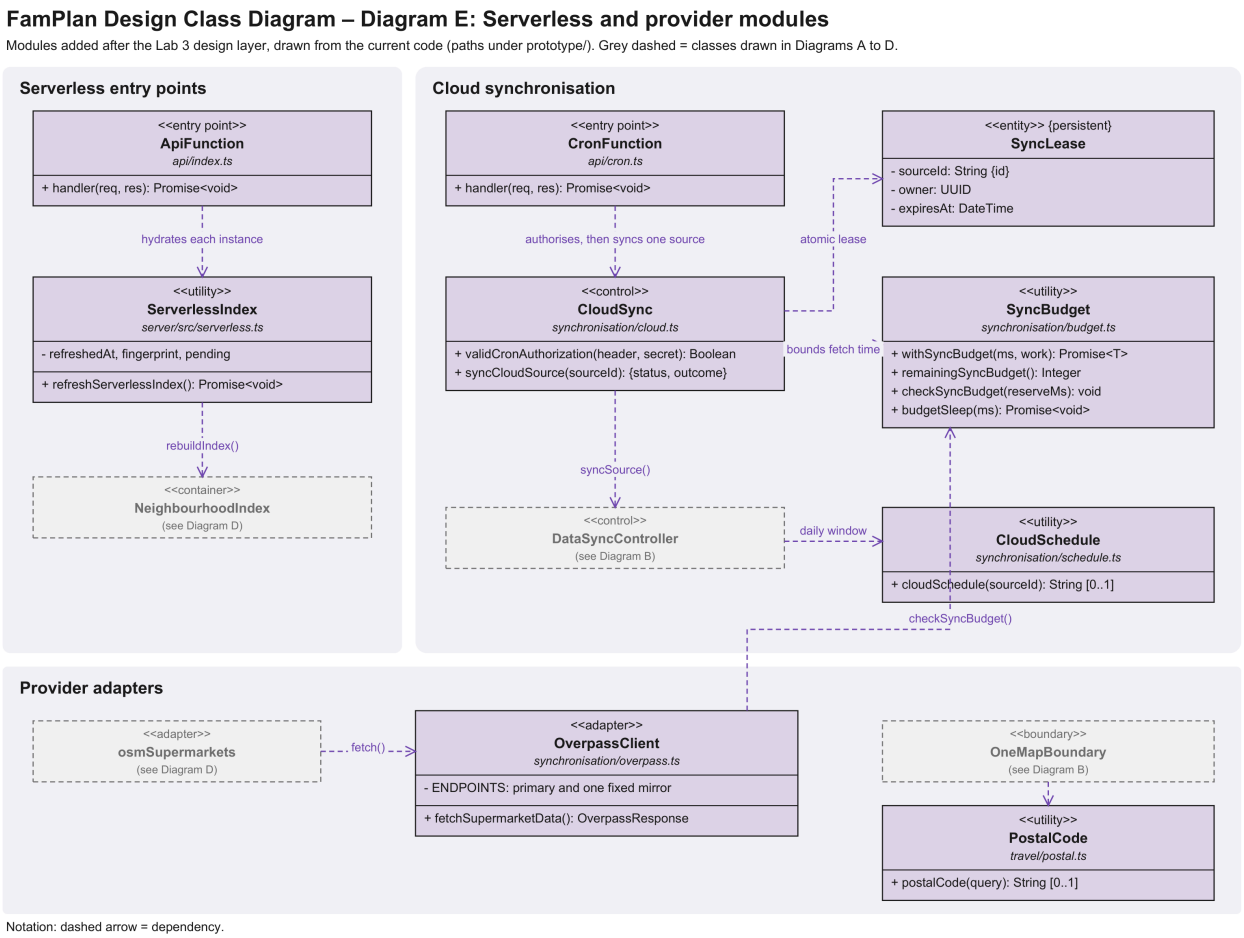


Diagram E below shows the modules added after Lab 3: serverless entry points, cron coordination with a database lease, the Overpass failover adapter and postal code handling. The commercial billing and workspace modules were removed when the project narrowed to personal family planning, so they are not modelled.



9.5 Design-to-Code Traceability

The prototype backend is organised by feature module (`prototype/server/src/<feature>/`) rather than by layer. Inside each module, `routes.ts` is the HTTP boundary, the service and logic files hold the control logic, and the Prisma models in `prisma/schema.prisma` hold the entities. The tables below map each design class in Sections 9.1, 9.2 and 9.4 to the code that implements it; paths are relative to `prototype/` unless shown as `server/src`-relative.

Table 9.5.1 Boundary classes

Design class	Implemented in
AccountBoundary	`client/src/pages/SignIn.tsx, PasswordReset.tsx, Account.tsx`
PreferenceBoundary	`client/src/pages/Preferences.tsx`
DiscoveryBoundary	`client/src/pages/Explore.tsx`
NeighbourhoodBoundary	`client/src/pages/Profile.tsx`
MapRouteBoundary	`client/src/pages/Commute.tsx, components/map.tsx`
DecisionSupportBoundary	`client/src/pages/Compare.tsx, Shortlist.tsx, components/ScoreBreakdown.tsx`

Design class	Implemented in
AdminMonitoringBoundary	`client/src/pages/Admin.tsx`
EmailDeliveryBoundary	`server/src/account/mailer.ts` – sendMail()
OneMapBoundary	`server/src/travel/onemap.ts` – searchAddress(), geocodeCached(), onemapRoute()
PublicDataSourceBoundary	`server/src/synchronisation/sources.ts, datagovsg.ts` (SourceDefinition adapters)
(all UI boundaries to server)	`client/src/lib/api.ts` – api<T>(path) over JSON/HTTPS

Table 9.5.2 Control classes

Design class	Module (server/src/)	Key functions / endpoints
AccountController	`account/routes.ts, passwords.ts`	POST /auth/register, /auth/login, /auth/logout, /auth/forgot-password, /auth/reset-password; PATCH, DELETE /account; Argon2id hashPassword()
PreferenceController	`preferences/routes.ts, presets.ts`	GET, PUT /preferences; POST /preferences/suggest-weights; suggestWeights()
DiscoveryController	`neighbourhoods/service.ts, routes.ts`	GET /neighbourhoods – searchNeighbourhoods(), nameMatches()
NeighbourhoodController	`neighbourhoods/service.ts, routes.ts`	GET /neighbourhoods/:id, /:id/facilities – neighbourhoodDetail(), amenitySummaries(), facilitiesNear()
RouteController	`travel/route.ts, neighbourhoods/routes.ts`	GET /neighbourhoods/:id/route – routeEstimate(); GET /geocode
ComparisonController	`neighbourhoods/routes.ts`	GET /compare (compareQuerySchema: 2–4 ids) – summarise() per item
ScoringController	`scoring/score.ts, config.ts`	calculateScore(evidence, weights, datasetVersions, config, now), normalise(), roundHalfUp()
ShortlistController	`shortlist/routes.ts`	GET, POST /shortlist; DELETE /shortlist/:id; PUT /shortlist/:id/note
AdminMonitoringController	`synchronisation/routes.ts`	GET /admin/sources, /admin/history, /admin/metrics (behind requireAdmin)
SyncScheduler	`synchronisation/scheduler.ts`	startScheduler() (node-cron, SYNC_CRON), nextRunAt(); api/cron.ts (Vercel Cron in production)
DataSyncController	`synchronisation/pipeline.ts, validate.ts`	syncAll(), syncSource(), ensureRegistry(), onDatasetsChanged()
CloudSync	`synchronisation/cloud.ts, budget.ts, schedule.ts`	validCronAuthorization(), syncCloudSource(), withSyncBudget(), cloudSchedule()
OverpassClient	`synchronisation/overpass.ts`	fetchSupermarketData()
PostalCode	`travel/postal.ts`	postalCode()

Design class	Module (server/src/)	Key functions / endpoints
ApiFunction, CronFunction	`serverless.ts; prototype/ api/index.ts, api/cron.ts`	handler(), refreshServerlessIndex()

Table 9.5.3 Entity classes and persistence decisions

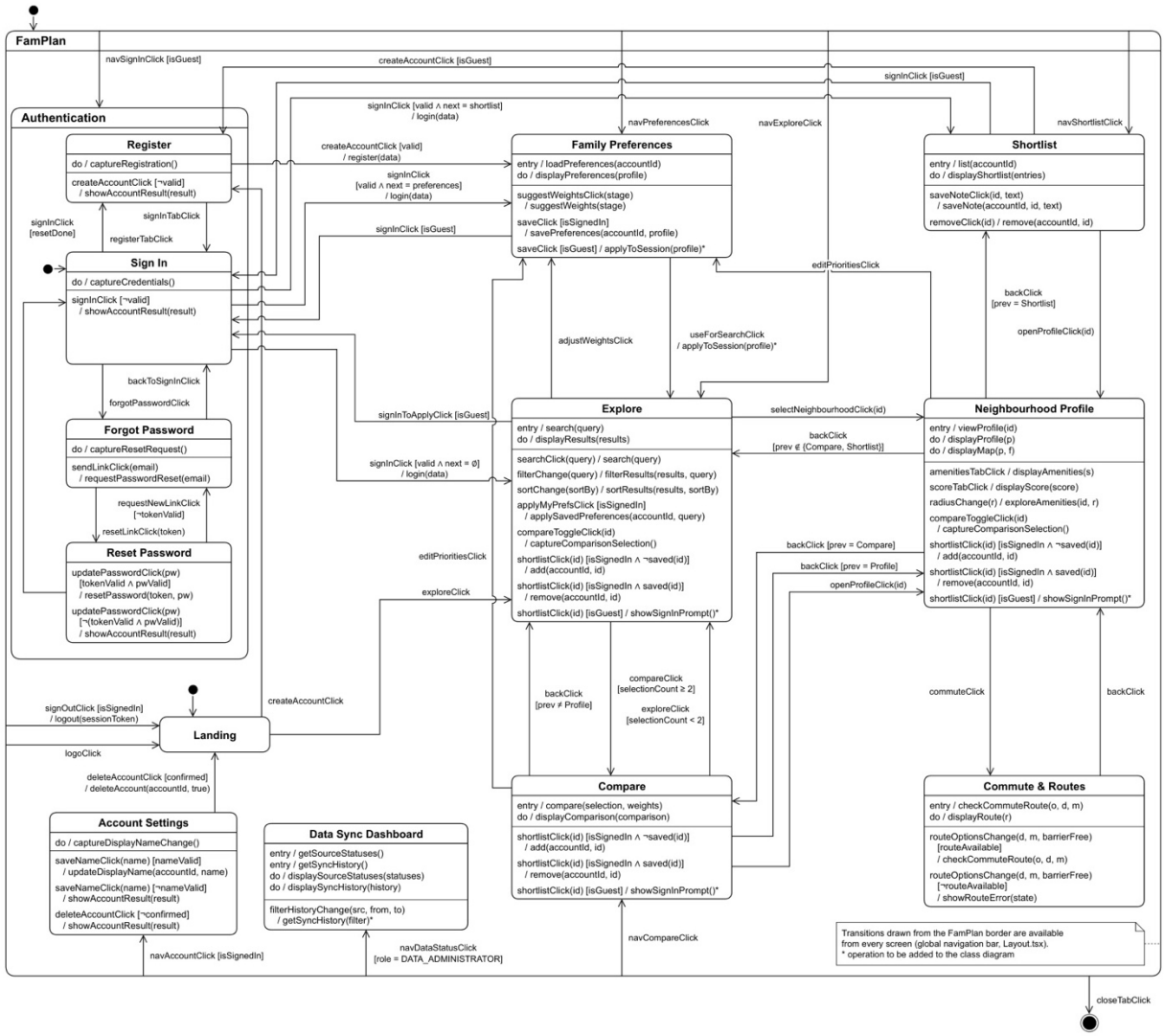
Design class	Decision	Stored as
UserAccount, PasswordCredential, Session, PasswordResetToken	{persistent}	Prisma models of the same name
FamilyPreferenceProfile, PriorityDestination	{persistent}	Prisma models of the same name
CriterionWeight, AmenityPreference	{persistent, embedded}	FamilyPreferenceProfile.w* columns; amenityThresholds (JSON)
NeighbourhoodProfile	{persistent}	Prisma model **Neighbourhood**
Facility	{persistent}	Prisma model Facility (one set per dataset snapshot)
ShortlistEntry, ShortlistNote	{persistent} / {persistent, embedded}	ShortlistEntry; the note is ShortlistEntry.note (max 500 characters)
DataSourceRegistry, SyncHistoryRecord	{persistent}	Prisma models of the same name
ValidatedDatasetSnapshot	{persistent}	Prisma model **DatasetSnapshot**
GeocodeCache (design only)	{persistent}	Prisma model GeocodeCache (cached OneMap postal lookups)
DataProvenanceStamp	{derived}	Built in NeighbourhoodIndex from DatasetSnapshot and DataSourceRegistry
ScoringConfiguration	{constant, versioned}	`scoring/config.ts` SCORING_CONFIG (version string)
SearchQuery, Geolocation, RouteEstimate, FacilityMatch, AmenitySummary	{transient}	Shared DTOs and Zod schemas in `shared/src`
FamilySuitabilityScore, CriterionScoreBreakdown, MissingDataTreatment, NeighbourhoodComparisonSet	{transient}	Computed per request (calculateScore(), / compare)

Table 9.5.4 Design-layer classes (Section 9.4)

Design class	Role (Lab 3 heuristic)	Implemented in (server/src/)
ServerBootstrap	Start-up class	`index.ts, app.ts, config.ts`
SessionAuthMiddleware	Access control	`account/sessions.ts` – loadSession, requireUser, requireAdmin
PrismaClient «singleton»	Persistent data	`db.ts, prisma/schema.prisma`
NeighbourhoodIndex	Container class	`neighbourhoods/store.ts`
SourceCatalogue	Container class	`synchronisation/sources.ts` – SOURCES

Design class	Role (Lab 3 heuristic)	Implemented in (server/src/)
SourceDefinition «adapter» + 9 adapters	Adapter pattern	`synchronisation/sources.ts, datagovsg.ts`
NormalisationRule «strategy»	Strategy pattern	`scoring/config.ts, score.ts`
DatasetValidator	Utility	`synchronisation/validate.ts`
RequestValidation	Utility (input validation, errors)	`lib/http.ts, shared/src/schemas.ts`
GeoUtils	Utility	`lib/geo.ts`

10. Dialog Map



The screens above match the application's routes. The exact-address search flow adds the transitions below, and travel modes are saved through the existing Family Preferences saveClick.

State	Trigger	Result
Any screen	quickSearchClick(query)	Opens Explore and runs search(query).
Explore	searchClick(query) [one place matches]	search(query, place): counts and scores use that exact coordinate.
Explore	searchClick(query) [several places match]	showLocationChoices(places), and no result is chosen automatically.
Explore	chooseLocationClick(place)	search(query, place)
Explore	selectNeighbourhoodClick(id, origin)	Neighbourhood Profile opens using the selected address and radius.
Neighbourhood Profile	commuteClick(origin)	Commute & Routes starts its route from the same address.